

DOCUMENTACIÓN TÉCNICA

NexusMQ Console

Consola web de administración y monitorización
SPA React · BFF NestJS

Andrés Ojeda Rodríguez · 2026

Tabla de contenidos

Documentación técnica

Prefacio	10. Tiempo real: terminación de SSE
1. Resumen ejecutivo	11. Observabilidad y métricas
2. Contexto y motivación	12. Visualización
3. Glosario	13. Seguridad
4. Vista de conjunto	14. Modelo de errores
5. Principios de diseño	15. Estrategia de pruebas
6. Arquitectura del frontend	16. Puerta de calidad y CI/CD
7. Arquitectura del BFF	17. Despliegue
8. Contrato y tipos generados	18. Configuración y operación
9. Autenticación y sesiones	19. Limitaciones y trabajo futuro

Apéndices

Apéndice A. Registro de decisiones y evolución	Apéndice B. Bibliografía
--	--------------------------

Prefacio

Este documento es la **documentación técnica final** de NexusMQ Console: la referencia autoritativa del *qué*, el *porqué* y el *cómo* de la consola, organizada para leerse de principio a fin o consultarse por partes.

Qué es este documento

NexusMQ Console es la consola web de administración y monitorización del broker de mensajería distribuido [NexusMQ](#). Es un **cliente externo y remoto**: no toca el core C++ del broker, sino que consume su **plano de operación REST** (contrato OpenAPI) desde una arquitectura TypeScript de dos piezas —una **SPA** React y un **BFF** NestJS— empaquetadas en una sola imagen.

Como el broker que administra, es un **proyecto de aprendizaje y portfolio**. Su tesis no es acumular pantallas, sino demostrar que un cliente de operación serio se construye con las mismas exigencias que un backend: contrato como fuente de verdad, frontera de confianza explícita, degradación honesta, y una visualización que comunique el estado real del sistema en lugar de decorarlo.

Esta documentación recoge la consola **tal como está construida** (*as-built*): la visión y el contexto, la arquitectura de sus tres paquetes, el contrato y su generación de tipos, la autenticación confinada en servidor, el tiempo real, la observabilidad, la visualización, la seguridad, la estrategia de pruebas, la operación y el registro de decisiones que la llevó hasta aquí.

A quién va dirigido

- **Al autor**, como mapa de construcción y memoria del diseño.
- **A revisores técnicos** (entrevistadores, colaboradores) que evalúen el proyecto como muestra de competencia *full-stack*: TypeScript de extremo a extremo, arquitectura cliente-servidor con una frontera de confianza real, integración con un sistema distribuido ajeno a través de su contrato, y diseño de información para operar infraestructura.

Se asume familiaridad con desarrollo web moderno (React, HTTP, TypeScript) y con conceptos de operación (métricas, series temporales, autenticación por token). Los términos de dominio —los del broker y los de la consola— se definen en el [Glosario](#).

Cómo está organizado

La documentación se divide en **siete partes** más **apéndices**:

Parte	Contenido
I — Introducción y visión	Resumen ejecutivo, contexto y motivación, glosario.
II — Arquitectura	Vista de conjunto, principios de diseño, arquitectura del frontend y del BFF.
III — Contrato e integración	Contrato y tipos generados, autenticación y sesiones, tiempo real (SSE), observabilidad y métricas.
IV — Experiencia y seguridad	Visualización, seguridad, modelo de errores.
V — Calidad	Estrategia de pruebas, puerta de calidad y CI/CD.
VI — Operación y despliegue	Despliegue, configuración y operación.
VII — Evolución	Limitaciones y trabajo futuro.

Los **apéndices** recogen el registro de decisiones as-built y la bibliografía.

Cada capítulo vive en su propio fichero Markdown numerado bajo `docs/tecnica/`. Los **diagramas** son bloques **Mermaid embebidos** en el capítulo que los explica: no hay catálogo aparte, porque un diagrama de esta consola casi nunca se entiende sin la prosa que lo rodea.

Relación con el contrato del broker

La consola tiene **una sola fuente de verdad externa**: el `openapi.yaml` de NexusMQ. De él se **generan** los tipos y el cliente HTTP de `packages/contract`; nada del contrato se escribe a mano. Este documento **explica y enlaza** ese contrato —da *rationale*, flujos y diagramas— pero **no lo duplica**: para el detalle preciso de un *endpoint* o de un esquema, la referencia es el propio OpenAPI vendorizado en `packages/contract/openapi.yaml` y, aguas arriba, la [documentación técnica de NexusMQ](#).

Hay una segunda dependencia de contrato que **no** viaja por el OpenAPI: los **nombres de las métricas**. El OpenAPI fija la *forma* del snapshot (una lista abierta de muestras), no sus nombres; esos viven en el catálogo `docs/metrics.md` del broker. El [capítulo 11](#) explica cómo se ancla la consola a ese catálogo y qué pasa cuando el broker aún no emite una familia.

Alcance y fuentes

Esta documentación es la **fuentes de verdad** del proyecto. El código en `apps/` y `packages/` y la suite de pruebas son la referencia última cuando el detalle de implementación importa: todo lo que aquí se afirma está sostenido por código que compila y por tests que pasan. Cuando algo **no** funciona o está limitado, se dice —ver el [capítulo 19](#).

1. Resumen ejecutivo

Qué es NexusMQ Console, qué resuelve, cómo está construida y qué la distingue de una pantalla de administración cualquiera. Los capítulos siguientes desarrollan cada punto.

1.1 Qué es

NexusMQ Console es la **consola web de operación** del broker NexusMQ: una aplicación que permite ver el estado del clúster en vivo, administrar *topics*, inspeccionar grupos de consumo y particiones, seguir el consenso Raft y consultar la historia de métricas, todo contra un broker real y remoto.

Es **TypeScript de extremo a extremo** en un monorepo pnpm + Turborepo con tres paquetes:

Paquete	Qué es	Responsabilidad
apps/web	SPA React 19 sobre Vite	Interfaz, visualización, estado de cliente.
apps/bff	BFF NestJS 11 sobre Express	Frontera de confianza: proxy, sesiones, SSE, servido de la SPA.
packages/contract	Tipos + cliente generados	Única traducción del OpenAPI de NexusMQ a TypeScript.

1.2 El problema que resuelve

Un broker distribuido expone un plano de operación REST y un endpoint de métricas. Eso basta para `curl`, pero no para **operar**: entender si el clúster está sano exige correlacionar throughput, latencias por percentil, errores, conexiones y estado de consenso, y hacerlo mientras cambian. La consola convierte ese plano en una superficie de operación:

- **Dashboard vivo** — throughput *produce/fetch*, latencias p50/p99/p999 derivadas del histograma del intervalo, tasa de error, conexiones por plano y salud del clúster, todo empujado por **SSE** con fallback a *polling*.
- **Topics** — listado paginado, creación, descripción (configuración + particiones), edición de retención vía `PATCH` y borrado.
- **Grupos** — listado con estado y descripción con miembros, *offsets* confirmados y **lag real por partición**.
- **Particiones** — líder, *high-watermark*, época y **lag de réplica**, cruzando la descripción del topic con el estado Raft del clúster.
- **Cluster / Raft** — nodos, roles, término, *commit index* y una **topología 3D** que responde a los cambios de líder.
- **Historia** — series temporales desde Prometheus con ventanas de 15 min a 24 h.

1.3 Por qué existe el BFF

El navegador **nunca** habla directamente con el broker. Entre ambos hay un BFF NestJS que existe por cuatro razones concretas, no por moda arquitectónica:

1. **Confinar el JWT**. El token del broker se guarda en servidor; al navegador solo viaja una cookie de sesión `httpOnly` firmada. Hay tests que verifican que el token no aparece nunca en una respuesta ([capítulo 9](#)).

2. **Terminar el SSE.** El BFF mantiene su propia conexión al *stream* del broker y reconecta con *backoff + jitter* sin tumbar el `EventSource` del navegador ([capítulo 10](#)).
3. **Cerrar la superficie de consulta.** El endpoint de historia no acepta PromQL cruda: el cliente elige un **id de una allow-list** y el BFF construye la consulta ([capítulo 11](#)).
4. **Servir la SPA en el mismo origen.** Sin CORS, con CSP anclada a `'self'` ([capítulo 13](#)).

1.4 Qué la distingue

- **El contrato no se escribe a mano.** `openapi-typescript` genera los tipos desde el `openapi.yaml` vendorizado del broker; un cambio de contrato rompe el `typecheck`, no el *runtime* ([capítulo 8](#)).
- **Degradación honesta.** Si el broker no emite una familia de métricas, la consola muestra «—» y lo dice; no inventa ceros. Si no hay Prometheus, la vista de Historia avisa y el resto sigue funcionando.
- **Un solo modelo de error.** RFC 7807 de punta a punta: el broker lo emite, el BFF lo propaga *verbatim* o genera el suyo, y la SPA lo muestra con icono y texto ([capítulo 14](#)).
- **Visualización con criterio.** Cuatro librerías elegidas por lo que cada una hace mejor (uPlot para *streaming*, ECharts para exploración, visx para composición, react-three-fiber para la topología), todas alimentadas por **un único sistema de tokens** validado en claro y oscuro ([capítulo 12](#)).
- **Verificación en navegador de verdad.** 21 pruebas Playwright, de las cuales 14 corren contra la topología de producción real —el BFF sirviendo la SPA y proxyando a un doble del broker— y no contra mocks de red ([capítulo 15](#)).

1.5 Estado

Versión 1 completa y cerrada. Cinco fases entregadas: andamiaje del monorepo, BFF, SPA base y sistema de diseño, vistas v1, historia y empaquetado, y una fase final de reconciliación con el broker real que corrigió el catálogo de métricas y endureció el servidor.

Indicador	Valor
Pruebas unitarias y de integración	105 (BFF 82 · web 22 · contract 1)
Pruebas e2e en navegador	21 (SPA 7 · full-stack 14)
Puerta de calidad	install · generate · lint · format · typecheck · build · test
Empaquetado	Imagen Docker multi-stage, no-root, con <code>HEALTHCHECK</code>

El detalle de lo que **no** hace y por qué está en el [capítulo 19](#).

2. Contexto y motivación

Por qué un broker necesita una consola, por qué esta consola es un proyecto **separado** del broker, y qué tesis técnica sostiene. Es el capítulo del *porqué*.

2.1 El punto de partida: un broker sin cara

NexusMQ es un broker de mensajería distribuido en C++23 con arquitectura *shared-nothing thread-per-core* y **Raft por partición**. Expone tres planos bien separados:

- **plano de datos** — *produce/fetch* por protocolo binario nativo o subconjunto Kafka;
- **plano de consenso** — RPC de Raft entre réplicas;
- **plano de control** — API REST `/api/v1`, salud (`/healthz`, `/readyz`) y `/metrics`.

El plano de control es completo y está bien diseñado, pero es *máquina a máquina*. Operarlo significa encadenar `curl` y leer JSON, o exponer `/metrics` a Prometheus y montar tableros genéricos. Ninguna de las dos cosas responde bien a la pregunta que un operador se hace de verdad: **¿está sano esto, y si no, dónde duele?**

Responderla exige correlacionar, en la misma pantalla y al mismo tiempo:

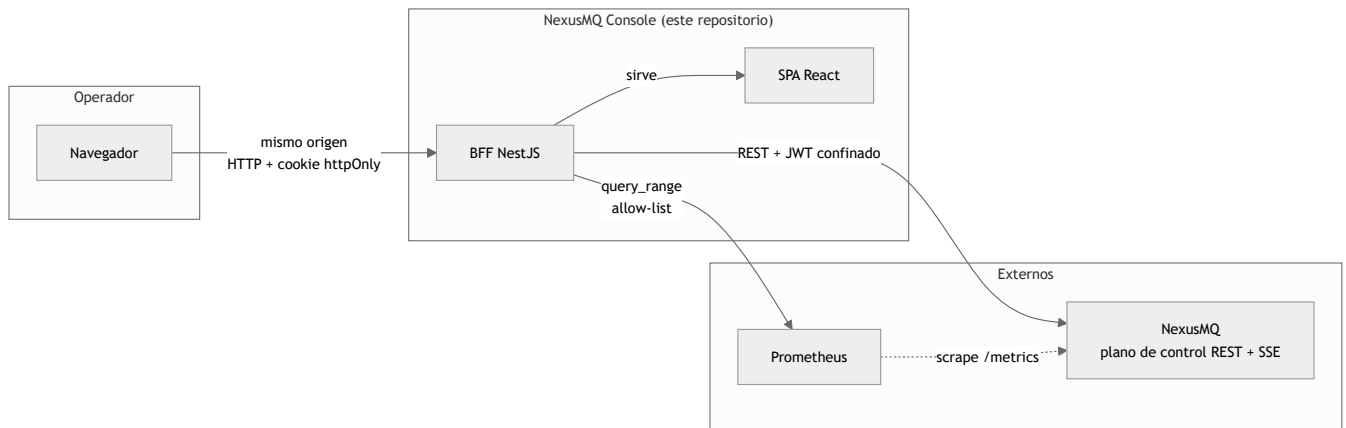
- el **throughput** por api (`produce / fetch`) y su tendencia;
- las **latencias por percentil**, que son lo único que describe la cola de la distribución;
- la **tasa de error** en proporción a las peticiones, no en absoluto;
- las **conexiones activas** por plano;
- el **estado del consenso**: quién lidera cada partición, con qué término, cuánto va retrasado cada seguidor.

Esa correlación es un problema de **diseño de información**, y es el que resuelve la consola.

2.2 Por qué un proyecto separado

La consola podría haber vivido dentro del repositorio del broker, servida por el propio `nexusd`. No se hizo, por cuatro razones:

1. **Disciplina de contrato.** Al ser un repositorio distinto, la consola **solo** puede apoyarse en lo que el broker publica: su `openapi.yaml` y su catálogo de métricas. No hay atajos por conocimiento interno. Si algo no está en el contrato, no existe. Esta restricción es deliberada y es la que mantiene honesta la integración.
2. **Ciclos de vida distintos.** El broker es C++ compilado, con *sanitizers* y dos *toolchains*; la consola es TypeScript con Vite y Playwright. Mezclar sus puertas de calidad habría penalizado a ambos.
3. **Superficie de ataque separada.** El broker no debería tener que servir HTML ni gestionar sesiones de navegador. Mantener la consola fuera deja el core libre de esa responsabilidad, coherente con su propia separación plano de datos / plano de control.
4. **Despliegue independiente.** La consola escala, se actualiza y se reinicia sin tocar el broker; puede apuntar a distintos despliegues sin recompilar nada.



2.3 La tesis técnica

Un cliente de operación se juzga por tres cosas, y la consola está construida alrededor de ellas:

Primera: la frontera de confianza tiene que estar en algún sitio concreto. En una SPA "pura" que llama al broker desde el navegador, el token del operador vive en `localStorage` y el broker queda expuesto a CORS y a cualquier XSS. Aquí la frontera es el **BFF**: es la única pieza que conoce la URL del broker y el único lugar donde vive el token. El navegador recibe una cookie opaca y nada más. Todo el capítulo 9 desarrolla esta decisión.

Segunda: el tiempo real no es "abrir un WebSocket". Es decidir qué pasa cuando el *upstream* cae, cuando el cliente es más lento que la fuente, y cuando el transporte deja de estar disponible. La consola responde a las tres: reconexión con *backoff* + *jitter* que no tumba la conexión del navegador, **backpressure acotado** que pausa la lectura del broker hasta que el socket del cliente drena, y caída a *polling* si el propio plano SSE muere (capítulo 10).

Tercera: la visualización es ingeniería, no decoración. Un color elegido a ojo puede ser invisible para un operador daltónico a las tres de la mañana. La consola usa un **sistema de tokens único** validado por contraste en claro y oscuro, señala el estado siempre con **icono + etiqueta** además de color, y usa la paleta categórica en **orden fijo** para que una serie tenga el mismo color en todas las pantallas (capítulo 12).

2.4 La lección más cara: los dobles pueden mentir

Merece capítulo propio porque marcó la última fase del proyecto. Durante las fases 1 a 4 la consola se desarrolló contra **dobles de prueba** del broker, y todos los tests pasaban. Al conectar contra el broker real, el Dashboard apareció **vacío**: cada métrica en «—».

La causa: el `MetricsSnapshot` del OpenAPI es una **lista abierta** de muestras con `name` libre. El contrato fija la *forma*, no los *nombres*. La consola había asumido nombres plausibles (`nexusmq_messages_in_total` ...) y los dobles emitían exactamente esos nombres. Los tests verificaban que la consola sabía leer sus propias invenciones.

El broker real emite otra cosa: familias `nexus_broker_*` desglosadas por etiquetas `{api, protocol}` y `{plane}`. Corregirlo obligó a remapear el Dashboard y la Historia, a introducir **filtrado por etiqueta** y agregación de histogramas por `1e`, y a realinear **todos** los dobles con el contrato real.

De ahí salen dos reglas que la consola aplica desde entonces:

- **Un doble solo vale si emite lo que emite el original.** Un doble alineado con la suposición del cliente no prueba nada.
- **Cuando el contrato deja algo abierto, hay que anclarlo explícitamente.** Los tres módulos que fijan nombres de métrica llevan un `@see` clicable al catálogo `docs/metrics.md` del broker: si el catálogo cambia, el punto a tocar está señalizado.

El registro completo de esta reconciliación está en el [apéndice A](#).

2.5 No-objetivos

Decidir qué **no** hace un proyecto es tan importante como decidir qué hace:

No-objetivo	Razón
Reimplementar una TSDB	Eso es Prometheus. La consola consulta, no almacena.
Embeber Grafana	Un <code><iframe></code> no es un producto; la visualización propia es el punto.
Endpoint "kill" o control de ciclo de vida de nodos	Requiere una capa de servicio del SO que el broker aún no expone.
Aplicación móvil nativa	La web responsiva cubre el caso; una app nativa duplicaría el trabajo sin añadir tesis.
Tocar el core C++ del broker	La consola es un cliente. Si falta un dato, el arreglo es del contrato.
Multi-broker desde la interfaz	Convertiría al BFF en un proxy abierto (SSRF). Ver capítulo 19 .

3. Glosario

Términos que aparecen a lo largo del documento, divididos entre los que vienen del **broker** (dominio de mensajería) y los que son propios de la **consola** (dominio de cliente web). Los identificadores de código van en inglés; la prosa, en español.

3.1 Del dominio del broker

Broker — Nodo de NexusMQ (`nexusrd`). Sirve el plano de datos (produce/fetch), el plano de consenso (Raft) y el plano de control (REST `/api/v1` , `/healthz` , `/readyz` , `/metrics`).

Plano de operación / plano de control (*admin plane*) — La API REST de administración del broker, más salud y métricas. Es lo único que consume la consola. Escucha en el puerto que fija `--admin-port` .

Topic — Canal lógico de mensajes, dividido en particiones. La consola lo lista, crea, describe, altera (retención) y borra.

Partición — Unidad de paralelismo y de replicación de un topic. Cada partición es su propio grupo Raft en NexusMQ.

High-watermark (HWM) — Offset hasta el cual los registros de una partición están confirmados por el quórum y son visibles para los consumidores.

Leader epoch (época del líder) — Contador que identifica el mandato de un líder sobre una partición; permite detectar y truncar divergencias de log.

Raft — Algoritmo de consenso que NexusMQ ejecuta **por partición**. De él la consola muestra rol, líder, término (*term*), *commit index* y progreso de seguidores.

Follower lag (lag de réplica) — Cuánto va por detrás un seguidor respecto del líder de su partición. La consola lo obtiene del estado Raft (`GET /api/v1/cluster`), no del describe del topic.

Grupo de consumo — Conjunto de consumidores que se reparten las particiones de uno o varios topics. Tiene estado, generación, líder y miembros.

Consumer lag — Diferencia entre el *high-watermark* de una partición y el offset confirmado por el grupo: cuántos mensajes le faltan por consumir.

RFC 7807 (`application/problem+json`) — Formato estándar de documento de error que emiten tanto el broker como el BFF. Ver [capítulo 14](#).

3.2 De la consola

SPA (*Single Page Application*) — `apps/web` : la aplicación React que se ejecuta en el navegador. Servida por Vite en desarrollo y por el BFF en producción.

BFF (*Backend For Frontend*) — `apps/bff` : el servidor NestJS que existe **solo** para esta interfaz. Proxya el broker, confina el token, termina el SSE, expone el data source de historia y sirve la SPA. Es la frontera de

confianza.

Contrato — `packages/contract` : los tipos TypeScript y el cliente HTTP **generados** con `openapi-typescript` desde el `openapi.yaml` vendorizado del broker. Nunca se escribe a mano.

Vendorizado — Copia versionada del `openapi.yaml` del broker dentro de este repositorio, refrescable con `pnpm --filter @nexusmq/contract sync:openapi` . Hace el build reproducible y desacopla de la disponibilidad de la red.

Modo abierto / modo secreto — Los dos modos en que puede estar el broker. En **modo secreto** arrancó con `--jwt-secret` y exige `Authorization: Bearer` ; en **modo abierto**, no. El BFF descubre cuál es **sondeando el comportamiento** del broker, porque no conoce su secreto HS256. Ver [capítulo 9](#).

Sesión de operador — Entrada del almacén en memoria del BFF que guarda el JWT del broker que el operador pegó al iniciar sesión. Tiene TTL y se purga.

Cookie de sesión — `nexusmq_session` , `httpOnly` , `SameSite=Lax` , `Secure` en producción. Transporta `id.HMAC-SHA256(id, SESSION_SECRET)` : una clave opaca, nunca datos.

Gate de login (`CONSOLE_REQUIRE_LOGIN`) — Bandera del BFF que, activa (por defecto), exige sesión válida en las rutas protegidas **aunque el broker esté en modo abierto**.

@Protected() — Decorador que marca una ruta del BFF como sujeta al `SessionAuthGuard` . Las rutas sin él (salud, `stream` , endpoints de auth) pasan sin sesión.

Terminación de SSE — El BFF abre su propia conexión `text/event-stream` al broker y reemite los frames al navegador. El navegador nunca abre un `EventSource` contra el broker.

Backpressure acotado — Política del bombeo SSE: si el socket del cliente no acepta más datos, se **pausa la lectura del broker** hasta el evento `drain` , en lugar de acumular en memoria sin límite.

Degradación honesta — Regla de diseño transversal: cuando un dato no está disponible, la interfaz muestra «—» y explica por qué, en lugar de mostrar un cero que se lee como "todo tranquilo".

Allow-list de PromQL — Lista cerrada de ids de métrica de historia. El cliente envía un id; el BFF **construye** la PromQL en servidor. El endpoint no acepta consultas crudas.

Tokens dataviz — Variables CSS de `apps/web/src/styles/tokens.css` que definen superficies, tinta, estado y la paleta categórica de 8. Fuente única de color de la consola.

Doble (test double) — Servidor de prueba que implementa el contrato del broker o de Prometheus para los tests. Desde la fase 5, todos emiten los **nombres reales** del broker.

e2e full-stack — Suite Playwright que arranca la topología de producción real (BFF sirviendo la SPA + doble del broker + doble de Prometheus) y la ejerce desde el navegador.

4. Vista de conjunto

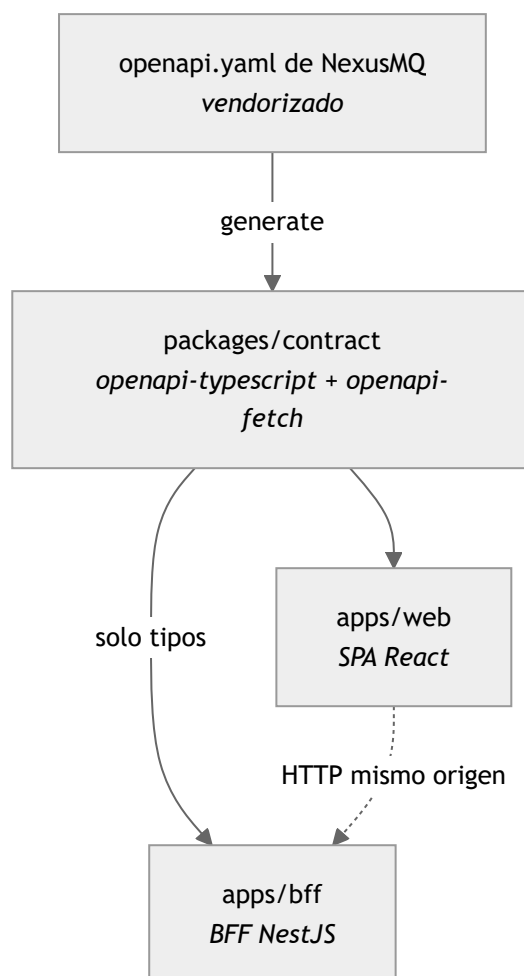
El sistema a vista de pájaro: los tres paquetes, cómo fluyen sus dependencias, qué recorre una petición de punta a punta y cómo se orquesta el monorepo. Es el mapa que sitúa los capítulos siguientes.

4.1 Tres paquetes y una dirección de dependencia

El repositorio es un monorepo **pnpm workspaces + Turborepo** con tres paquetes:

apps/web	SPA React (Vite)	→ depende de contract
apps/bff	BFF NestJS (Express)	→ depende de contract (solo tipos)
packages/contract	tipos + cliente generados	→ no depende de nada del proyecto

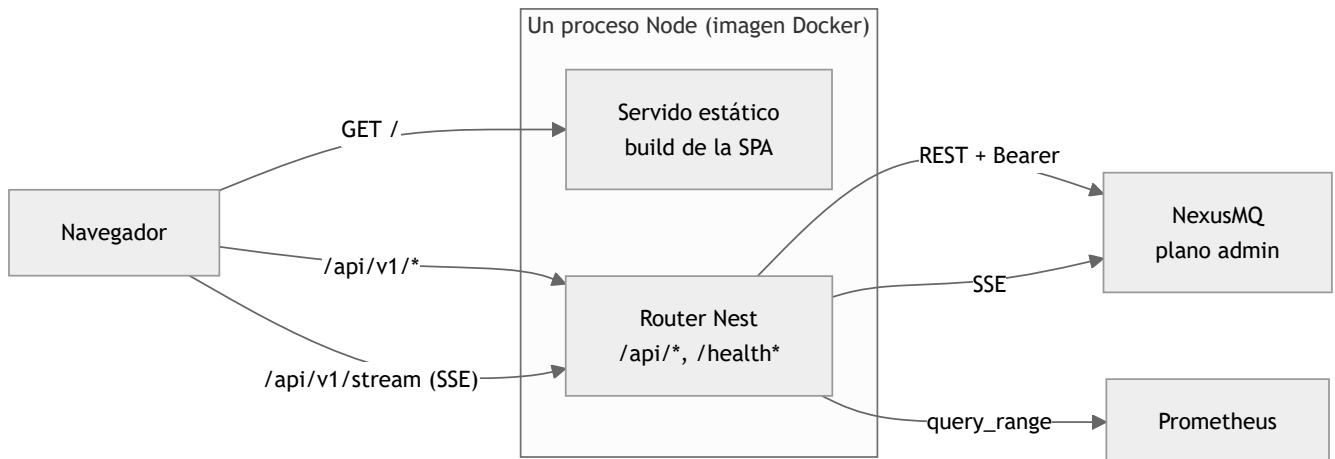
La regla es la misma que en el broker, trasladada a TypeScript: **las dependencias apuntan hacia el contrato**. `packages/contract` no conoce ni a la SPA ni al BFF; ambos lo consumen. No hay dependencia entre `apps/web` y `apps/bff` — se comunican por HTTP, no por `import`, y su acoplamiento es exactamente el contrato REST que el BFF expone.



En el BFF el contrato entra **solo como tipos**: el proxy usa el `fetch` de undici en modo *passthrough*, no el cliente `openapi-fetch`. La razón está en el [capítulo 7](#).

4.2 Topología de ejecución

En **producción** hay un único proceso Node: el BFF. Sirve la SPA ya construida como ficheros estáticos y expone la API en el mismo origen. El broker y Prometheus son externos y se apuntan por variables de entorno.



En **desarrollo** son dos procesos: Vite sirve la SPA con HMR en su puerto y proxya `/api/*` al BFF, que corre aparte con `nest start --watch`. El servidor estático del BFF es un *no-op* si `WEB_DIST_PATH` no apunta a un directorio con `index.html`, así que el mismo código vale para ambos modos sin ramas condicionales.

4.3 Mismo origen, sin CORS

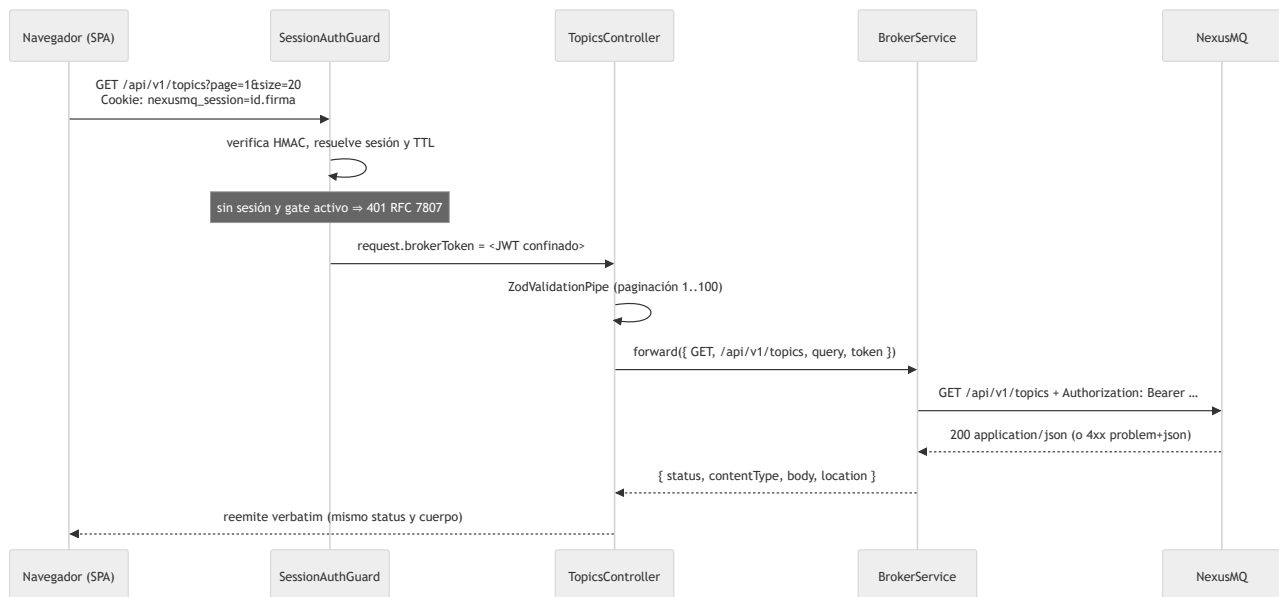
Decisión estructural: **la SPA y la API comparten origen**. El BFF no habilita CORS en ningún caso. Esto no es una simplificación, es una propiedad de seguridad:

- la cookie de sesión viaja sin necesitar `SameSite=None` ;
- el navegador bloquea por sí solo cualquier lectura desde otro origen;
- la CSP puede anclarse a `'self'` sin excepciones para `connect-src` .

El coste es que la consola debe desplegarse detrás de un único *hostname*. Es un coste que se acepta sin discusión: el capítulo 13 explica lo que compra.

4.4 Recorrido de una petición

Seguir una lectura de topics de punta a punta muestra dónde vive cada responsabilidad:



Cinco propiedades salen de este diagrama:

1. **El guard es global**, pero solo actúa sobre rutas marcadas con `@Protected()`.
2. **La validación ocurre en el borde**, con Zod y allow-list, antes de tocar la red.
3. **El token se inyecta server-side**; el navegador nunca lo ve.
4. **La respuesta del broker se reemite tal cual**: mismo status, mismo `Content-Type`, mismo cuerpo, incluido `Location` al crear. El BFF no reinterpreta el dominio del broker.
5. **El único error que el BFF fabrica** en este camino es el 502 de "broker inaccesible" y el 400 de validación; todo lo demás es del broker.

4.5 Mapa de módulos del BFF

Módulo	Responsabilidad
config	Esquema Zod del entorno, validación <i>fail-fast</i> . Es <code>@Global()</code> .
common	<code>ProblemDetailsFilter</code> (RFC 7807), <code>ZodValidationPipe</code> , <code>sendProxyResult</code> .
health	<code>GET /health</code> propio del BFF (liveness, sin tocar el broker).
broker	<code>BrokerService</code> (proxy undici) + controllers de topics, groups, cluster y observabilidad. Es <code>@Global()</code> .
auth	Sesiones, cookie firmada, <code>SessionAuthGuard</code> global, detección del modo del broker.
prometheus	Data source de historia con allow-list de PromQL.
stream	Terminación de SSE con reconexión y backpressure.

`BrokerModule` es `@Global()` y **no** importa `AuthModule`: así se rompe el ciclo que aparecería si el módulo de auth (que necesita el broker para validar tokens) y el de broker (que necesita el token de la sesión) se importaran mutuamente. El token llega al controller por el decorador de parámetro `@BrokerToken()`, que lo lee de la petición donde lo dejó el guard.

4.6 Mapa de carpetas de la SPA

`apps/web/src` está organizado **por feature**, no por tipo de fichero:

Carpeta	Contenido
<code>app/</code>	Composición: router, shell (sidebar/topbar), proveedor de TanStack Query, tema.
<code>features/</code>	Una carpeta por dominio: <code>auth</code> , <code>topics</code> , <code>groups</code> , <code>cluster</code> , <code>dashboard</code> , <code>history</code> , <code>metrics</code> , <code>live</code> , <code>settings</code> , <code>viz</code> .
<code>routes/</code>	Un componente de página por ruta; monta features, no contiene lógica.
<code>components/ui/</code>	Primitivos de diseño reutilizables (<code>Button</code> , <code>Card</code> , <code>Dialog</code> , <code>Input</code> , <code>ProblemAlert</code> , <code>Spinner</code>).
<code>lib/</code>	Cliente del contrato, normalización de errores RFC 7807, utilidades.
<code>styles/</code>	<code>tokens.css</code> (fuente única de color) y estilos globales.

Dentro de cada feature conviven su modelo puro (`.ts` sin React, testeable), sus hooks de datos (`use-*.ts`) y sus componentes. Los módulos puros — `metrics-snapshot.ts`, `history-range.ts` — concentran la lógica que merece pruebas unitarias y no arrastran DOM.

4.7 Orquestación del monorepo

`turbo.json` declara el grafo de tareas:

Tarea	Depende de	Notas
<code>generate</code>	—	Entrada <code>openapi.yaml</code> , salida <code>src/generated/**</code> .
<code>build</code>	<code>^build</code> , <code>generate</code>	Salida <code>dist/**</code> .
<code>typecheck</code>	<code>^build</code> , <code>generate</code>	Sin los tipos generados no hay typecheck válido.
<code>test</code>	<code>^build</code> , <code>generate</code>	Idem.
<code>dev</code>	—	<code>persistent</code> , sin caché, con <code>passThroughEnv</code> explícito.

El `passThroughEnv` de `dev` no es cosmético: Turborepo 2.x usa *env-mode* estricto y filtra las variables no declaradas. Sin él, el BFF arrancaba con el entorno vacío y abortaba en la validación de configuración. Las nueve variables que necesita están declaradas ahí.

5. Principios de diseño

Las reglas que gobiernan las decisiones de toda la consola. Cada una se aplica en varios capítulos; aquí se enuncian una sola vez, con su *rationale* y su coste.

5.1 El contrato es la fuente de verdad, y no se escribe a mano

Todo tipo que describa un dato del broker **se genera**. `openapi-typescript` produce `packages/contract/src/generated/schema.ts` desde el `openapi.yaml` vendorizado, y de ahí salen las respuestas, los cuerpos, los parámetros y hasta el `ProblemDetail` de los errores.

La consecuencia práctica es que **un cambio de contrato rompe la compilación, no la producción**. Si el broker renombra un campo, `pnpm typecheck` falla en cada punto que lo usaba. Es exactamente donde queremos que falle.

El coste: hay que resistir la tentación de "ajustar a mano un tipito". El directorio generado está en `.gitignore` precisamente para que nadie lo edite y para que el build siempre lo regenere.

5.2 El BFF es la frontera de confianza

No es una capa de conveniencia: es **la** decisión de seguridad del proyecto. Todo lo que importa está de su lado:

- la **URL del broker** (el navegador no la conoce);
- el **JWT del operador** (vive en un `Map` en servidor, con TTL);
- la **construcción de la PromQL** (el cliente solo elige un id de una lista cerrada);
- la **validación de entrada** (Zod, allow-list, antes de tocar la red).

La regla derivada es sencilla de auditar: **si un dato viaja al navegador, es porque el navegador puede verlo sin riesgo**. La consola tiene tests que lo comprueban sobre el caso más sensible —el token del broker no aparece en el cuerpo, ni en las cabeceras, ni en la cookie de ninguna respuesta.

5.3 Inversión de dependencias en ambos lados

En el BFF, la inversión es la de NestJS: **DI por constructor**, servicios detrás de interfaces implícitas, controllers finos que no saben *cómo* se habla con el broker, solo *qué* piden. `BrokerService` encapsula undici, el dispatcher TLS y la construcción de URL; si mañana el transporte cambia, el cambio no sale de ese fichero.

En la SPA, la inversión toma otra forma: **la lógica pura no depende de React**. Los módulos que calculan cuantiles, derivan tasas, construyen ventanas temporales o alinean series son funciones puras en ficheros sin `import React`. Los hooks (`use-*.ts`) son adaptadores delgados que conectan esa lógica con TanStack Query y con el ciclo de vida del componente. Por eso el 100 % de las pruebas unitarias de la SPA son de lógica pura y ninguna necesita montar un DOM.

5.4 Controllers finos, servicios con la lógica

Ningún controller del BFF contiene una decisión de negocio. El patrón, repetido en los cinco controllers de proxy, es siempre el mismo:

```
@Get()
async list(
  @Query(new ZodValidationPipe(paginationQuerySchema)) query: PaginationQuery,
  @BrokerToken() token: string | undefined,
  @Res() res: Response,
): Promise<void> {
  const result = await this.broker.forward({ method: 'GET', path: '...', query, token });
  sendProxyResult(res, result);
}
```

Validar en el borde, delegar, reemitir. Un controller que crece es la señal de que falta un servicio.

5.5 Passthrough: no reinterpretar el dominio ajeno

El BFF **no traduce** las respuestas del broker. Captura status, Content-Type , cuerpo y Location , y los reemite *verbatim*. No normaliza campos, no añade envolturas, no convierte un 409 en un 400 "más bonito".

La razón es que cualquier traducción introduce una segunda definición del dominio que puede divergir de la primera. Si el broker devuelve un 409 Conflict con un problem+json explicando que el topic ya existe, ese es el mejor error posible: el BFF no tiene información adicional que aportar y sí muchas formas de estropearlo.

La excepción es acotada y explícita: el BFF **sí** genera errores propios cuando el fallo es suyo —validación de entrada (400) o imposibilidad de contactar con el broker (502)— y cuando la ruta es suya (auth, historia). Ver [capítulo 14](#).

5.6 Degradación honesta

Cuando falta un dato, la interfaz lo **dice**. No hay ceros de relleno, no hay gráficas vacías sin explicación, no hay estados de carga infinitos.

Esto se aplica en tres niveles:

Situación	Comportamiento
Una familia de métricas que el broker no emite	El tile muestra «—» y su línea de apoyo también.
Prometheus no configurado	La vista de Historia muestra un aviso con el nombre de la variable que falta; el resto de la consola sigue igual.
Clúster de un solo nodo con RF=1	La topología muestra el nodo con su halo y el texto «sin particiones replicadas», no un error.
Sin grupos de consumo	Empty state explícito, no una tabla vacía.

El principio detrás: **un cero es una afirmación**. Mostrar `0 msg/s` cuando en realidad no sabemos el valor es mentir en la dirección más peligrosa, porque se lee como "todo tranquilo".

5.7 Fail-fast en la configuración

El BFF valida **todo** su entorno con un esquema Zod en el constructor de `ConfigService`. Si falta `BROKER_ADMIN_URL`, si `SESSION_SECRET` tiene menos de 32 caracteres o si una URL está mal formada, el proceso imprime qué clave falla y por qué, y sale con código 1.

Nunca arranca en un estado medio válido. Un orquestador ve el fallo inmediatamente en el primer arranque, no tres horas después cuando alguien intenta iniciar sesión.

5.8 Accesibilidad como restricción, no como repaso

Dos reglas duras, verificadas por tests:

- **Contraste AA ($\geq 4.5:1$)** del texto sobre página y sobre superficie, en claro **y** en oscuro. Hay una prueba Playwright que lo mide sobre el DOM real.
- **El color nunca es el único portador de información**. Cada estado lleva icono y etiqueta: el badge de salud, el estado de un grupo, el aviso de error, la tasa de error del Dashboard. En las gráficas, la leyenda nombra las series.

5.9 Verificar en el navegador, no solo en el test unitario

Una función pura verificada no prueba que la pantalla funcione. Cada feature con superficie visual se comprobó **arrancando la aplicación**: Playwright sobre el build de producción para la SPA, y sobre la **topología de producción completa** (BFF real sirviendo la SPA y proxyando a un doble del broker) para todo lo que cruza la red.

Es la razón de que la suite full-stack levante tres procesos y de que no haya un solo `page.route()` sustituyendo a la API en esas 14 pruebas, salvo donde el objetivo del test es precisamente forzar una respuesta degradada.

6. Arquitectura del frontend

Cómo está construida la SPA: stack, organización por features, enrutado, guard de acceso, gestión de estado y datos, y el sistema de diseño que la sostiene.

6.1 Stack y por qué

Pieza	Elección	Razón
Framework	React 19	Ecosistema de visualización más rico; el proyecto necesita ECharts, uPlot, visx y react-three-fiber conviviendo.
Bundler	Vite 8	Arranque instantáneo, HMR real, <i>code splitting</i> por <code>import()</code> sin configuración.
Estilos	Tailwind v4 (CSS-first)	Los tokens viven en CSS (<code>@theme inline</code>), no en un <code>tailwind.config.js</code> ; el tema se conmuta reasignando variables en un único fichero.
Primitivos	Radix UI	Diálogo, tooltip y toggle-group accesibles de serie (foco atrapado, <code>Escape</code> , ARIA). No se reimplementa accesibilidad.
Enrutado	React Router 7 (data router)	Permite añadir <i>loaders</i> /acciones más adelante sin reestructurar.
Datos	TanStack Query	Caché, invalidación, reintentos y estados de carga/error resueltos; el hueco exacto que deja <code>openapi-fetch</code> .
Iconos	lucide-react	Consistente y tree-shakeable; los iconos son parte del sistema de estado, no decoración.

6.2 Organización por feature

`src/` se divide por **dominio**, no por tipo de artefacto:

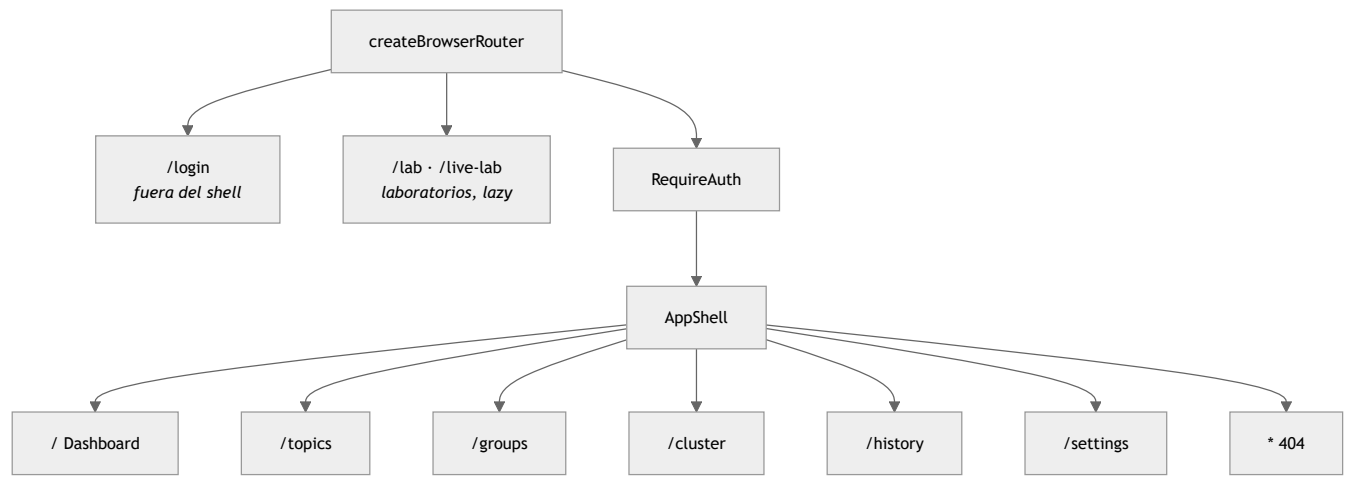
```
app/          router, shell (sidebar + topbar), QueryProvider, ThemeProvider
features/     auth · topics · groups · cluster · dashboard · history
              metrics · live · settings · viz
routes/       una página por ruta; monta features, sin lógica propia
components/   ui/ — primitivos de diseño (Button, Card, Dialog, Input, ProblemAlert, Spinner)
lib/          api-client (contrato → BFF), problem (RFC 7807), cn
styles/       tokens.css (fuente única de color), global.css
```

Dentro de una feature conviven tres tipos de fichero, y la separación es deliberada:

- **modelo puro** (`metrics-snapshot.ts` , `history-range.ts` , `format.ts`) — funciones sin React ni red. Es donde vive la lógica que merece pruebas unitarias.
- **hooks de datos** (`use-topics.ts` , `use-live-metrics.ts` , `use-cluster.ts`) — adaptadores entre el modelo puro, TanStack Query y el ciclo de vida del componente.
- **componentes** (`topics-table.tsx` , `stat-tile.tsx`) — presentación.

Un componente que calcula un cuantil es un error de capa; ese cálculo pertenece al modelo puro, donde se puede probar sin montar un DOM.

6.3 Enrutado y guard de acceso



`/login` vive **fuera** del shell (no tiene sentido mostrar navegación a quien no ha entrado). Los dos laboratorios — `/lab` (arsenal de visualización) y `/live-lab` (SSE con fallback)— quedan también fuera del guard y se cargan con `lazy()`, para que ECharts y three.js no entren en el *bundle* principal.

Todo lo demás pasa por `RequireAuth`, que resuelve el **estado de acceso** antes de montar nada:

Estado	Significado	Comportamiento
<code>authenticated</code>	Hay sesión de operador válida	Deja pasar.
<code>open</code>	El broker no exige auth y el gate está desactivado	Deja pasar sin login.
<code>locked</code>	Hace falta iniciar sesión	Redirige a <code>/login</code> , recordando la ruta de origen.

Mientras resuelve muestra un *spinner* con etiqueta; si falla, un `ProblemAlert` con botón de reintento. Nunca una pantalla en blanco.

6.4 Datos: TanStack Query sobre el cliente del contrato

Toda lectura de la API pasa por `apiClient`, el cliente `openapi-fetch` del contrato apuntado a `window.location.origin` — es decir, **al BFF**, mismo origen. Al ser mismo origen, la cookie `httpOnly` viaja sola con `credentials: 'same-origin'` y el JavaScript nunca la toca.

La política de reintentos del `QueryClient` codifica una distinción importante:

```

retry: (failureCount, error) => {
  if (error instanceof ProblemError && error.status >= 400 && error.status < 500) {
    return false; // 4xx: determinista, reintentar no cambia nada
  }
  return failureCount < 2; // red o 5xx: transitorio, merece reintento
}

```

Reintentar un 401 solo consigue tres 401. Reintentar un fallo de red puede salvar la vista.

`staleTime` es de 5 s por defecto y `refetchOnWindowFocus` está desactivado: en una consola de operación, un refresco inesperado al volver a la pestaña reordena tablas bajo el cursor.

Las mutaciones nunca actualizan la caché de forma optimista. Crear, alterar o borrar un topic invalida las claves afectadas y **se relee del broker**. En un sistema distribuido, el resultado real de una escritura puede no ser el que el cliente supuso; la única fuente de verdad es lo que el broker responda después.

6.5 Estado: tres ámbitos, tres mecanismos

Ámbito	Mecanismo	Ejemplo
Servidor (remoto)	TanStack Query	Lista de topics, describe de grupo, estado del clúster.
Flujo en vivo	<code>useLiveStream</code> + <code>useState</code> con ventana deslizante	Métricas del Dashboard (150 puntos).
Preferencias e interfaz	Contexto de React + <code>localStorage</code>	Tema claro/oscurito/sistema.

No hay una tienda global de estado. No hace falta: cada dato tiene un dueño claro y el acoplamiento entre features es mínimo.

El caso más interesante es el de las métricas en vivo. `useLiveMetrics` mantiene el frame anterior en una `ref` y, por cada frame nuevo, **deriva** una muestra: las tasas por diferencia de *counters* filtrados por etiqueta, y los cuantiles del histograma **del intervalo** (diferencia de cubos acumulativos). La derivación es pura y está probada; el hook solo orquesta. Ver [capítulo 11](#).

6.6 Tema y sistema de diseño

El tema se conmuta con un atributo `data-theme` en el `:root`, y los tokens se declaran en **dos ámbitos** para que el toggle del usuario gane siempre sobre la preferencia del sistema:

```

:root { /* valores light */ }

@media (prefers-color-scheme: dark) {
  :root:where(:not([data-theme='light'])) { /* dark del sistema */ }
}

:root[data-theme='dark'] { /* dark por toggle: gana siempre */ }

```

El `:not([data-theme='light'])` es lo que permite que un usuario con el sistema en oscuro fuerce el modo claro. Hay un script anti-FOUC en `index.html` que aplica el atributo antes del primer *paint* — y ese script inline es, precisamente, el que la CSP habilita por *hash* (ver [capítulo 13](#)).

Los componentes **no eligen color**: consumen roles (`bg-surface`, `text-foreground`, `text-critical`) mapeados a los tokens vía `@theme inline`. Cambiar la paleta es tocar un fichero.

6.7 Servido en desarrollo y en producción

En **desarrollo**, Vite sirve la SPA en el 5173 y proxya `/api`, `/health`, `/healthz` y `/readyz` al BFF en el 3000. Así el navegador ve un solo origen también en local, y la cookie de sesión se comporta igual que en producción.

En **producción**, `vite build` produce `dist/` y el BFF lo sirve (ver [capítulo 7](#)). No hay servidor web adicional.

El *bundle* se divide con `import()` diferido en las rutas de laboratorio y en la topología 3D, que arrastra `three.js`. El resto de la aplicación entra en el chunk principal.

6.8 Accesibilidad

Verificada, no declarada:

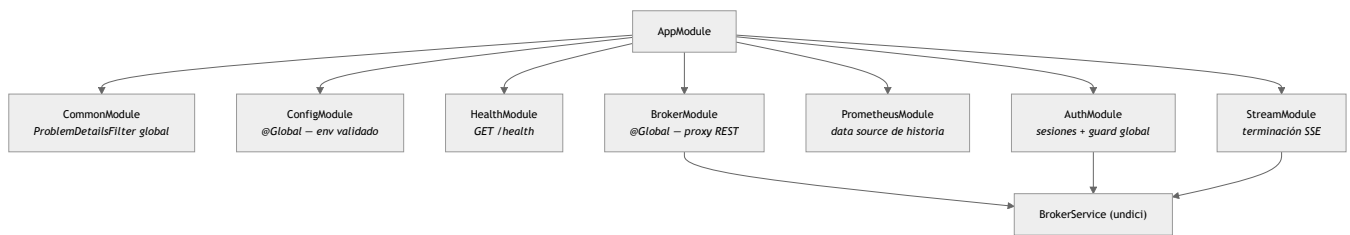
- **Contraste AA ($\geq 4.5:1$)** del texto sobre página y sobre superficie, en claro y en oscuro, medido por una prueba Playwright sobre el DOM real.
- **Estado nunca solo por color**: el badge de conexión, el estado de un grupo, la tasa de error y los avisos llevan icono + texto.
- **Navegación anunciada**: el enlace activo del sidebar lleva `aria-current`.
- **Gráficas etiquetadas**: cada wrapper de visualización expone `role="img"` con una etiqueta descriptiva, y las leyendas nombran las series.
- **Diálogos accesibles** por Radix: foco atrapado, cierre con `Escape`, bloqueo de *scroll*.

7. Arquitectura del BFF

El servidor NestJS que es la frontera de confianza de la consola: sus módulos, el proxy *passthrough* al plano admin del broker, el data source de Prometheus, el servido de la SPA y el arranque.

7.1 Composición

`AppModule` compone siete módulos, cada uno con una responsabilidad y ninguno con lógica de negocio del broker:



`BrokerModule` es `@Global()` y **no** importa `AuthModule`. Es la forma de romper un ciclo real: `auth` necesita el broker para validar un token, y los controllers del broker necesitan el token de la sesión. La dependencia se invierte con un decorador de parámetro — `@BrokerToken()` — que lee el token de la petición, donde lo dejó el guard. `auth` depende de `broker`; `broker` no depende de `auth`.

7.2 Tooling: por qué CommonJS y por qué undici

Dos decisiones del BFF se desvían del resto del monorepo, y ambas tienen razón:

CommonJS. `apps/bff` emite CommonJS mientras `apps/web` y `packages/contract` son ESM. NestJS depende de `reflect-metadata` y de `emitDecoratorMetadata` para su DI por constructor, y ese camino es el idiomático y el mejor soportado en CJS. Forzar ESM habría añadido fricción sin ganancia. En los tests, la metadata de decoradores la produce `unplugin-swc`.

fetch de undici, no openapi-fetch. El contrato entra en el BFF **solo como tipos**. El proxy es *passthrough*: no interpreta el cuerpo, lo reemite. Un cliente tipado que parsea la respuesta sería trabajo desperdiciado —y una oportunidad de divergencia. Además, undici da acceso al `Dispatcher`, necesario para el criterio TLS configurable y para el *streaming* del SSE.

7.3 El proxy: `BrokerService.forward()`

Un único método concentra todo el diálogo con el broker:

```
async forward(options: ForwardOptions): Promise<ProxyResult>
```

Recibe método, ruta, *query*, cuerpo y token opcional; devuelve `{ status, contentType, body, location }` — la respuesta **capturada, no interpretada**. El controller la reemite con `sendProxyResult`.

Detalles que importan:

- **Accept:** `application/json`, `application/problem+json` en toda petición: se aceptan explícitamente los dos formatos que el broker puede devolver.
- **Authorization:** `Bearer ...` solo si hay token. En modo abierto no se envía nada.
- **TLS configurable:** si `BROKER_TLS_REJECT_UNAUTHORIZED=false`, se construye un `Agent` de undici propio con `rejectUnauthorized: false`. En el caso estricto (por defecto) se usa el dispatcher global, sin crear objetos innecesarios.
- **Un único modo de fallo propio:** si `fetch` lanza (DNS, red, TLS, *timeout*), se convierte en `BrokerUnreachableError`, que el filtro global traduce a **502**. Cualquier respuesta del broker —incluidos 4xx y 5xx— **no** es un error del BFF y viaja intacta.

7.4 Controllers de proxy

Cinco controllers cubren el plano admin, todos con la misma forma:

Controller	Rutas	Protegido
TopicsController	GET/POST <code>/api/v1/topics</code> , GET/PATCH/DELETE <code>/api/v1/topics/{name}</code>	Sí
GroupsController	GET <code>/api/v1/groups</code> , GET <code>/api/v1/groups/{id}</code>	Sí
ClusterController	GET <code>/api/v1/cluster</code>	Sí
ObservabilityController	GET <code>/healthz</code> , GET <code>/readyz</code> , GET <code>/api/v1/metrics/snapshot</code>	Solo el snapshot
StreamController	GET <code>/api/v1/stream</code>	No (ruta abierta, como en el contrato)

`healthz` y `readyz` quedan abiertos porque son *probes* de orquestador: pedirle a Kubernetes que inicie sesión para comprobar la salud no tiene sentido. El **snapshot de métricas sí está protegido** — lo pasó a `@Protected` la fase 5, al detectarse que era alcanzable sin sesión incluso con el broker en modo secreto. Era una fuga de información operativa.

7.5 Validación en el borde

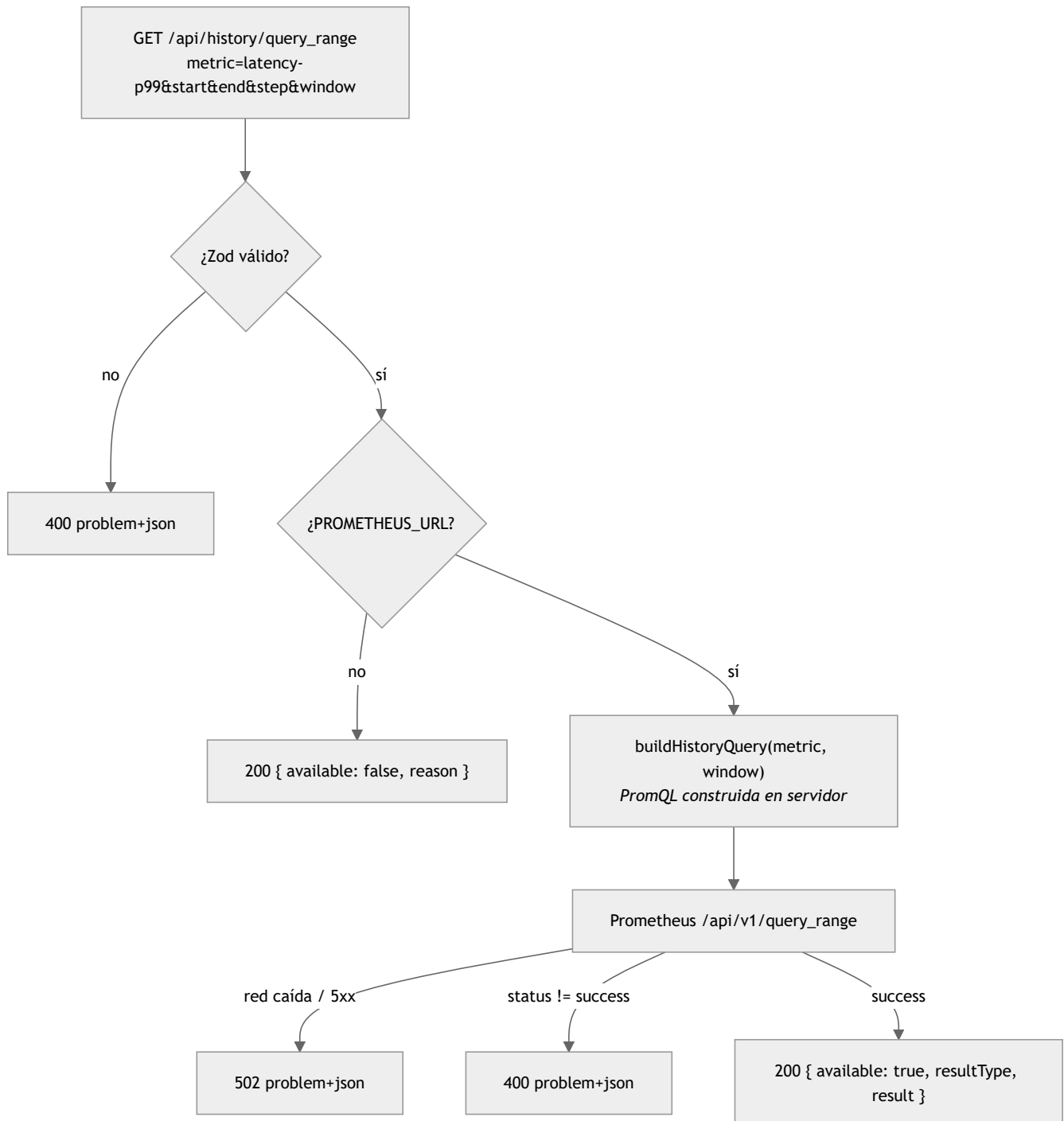
Toda entrada del cliente se valida con Zod antes de tocar la red, mediante `ZodValidationPipe` aplicado **por parámetro** (`@Query(pipe)`, `@Body(pipe)`, `@Param(pipe)`) en lugar de un pipe global sobre DTOs. Mantiene los controllers finos y hace explícito qué se valida en cada punto.

Entrada	Regla
Paginación	<code>page ≥ 1</code> , <code>size</code> en <code>[1, 100]</code> .
Nombre de topic	Obligatorio, no vacío.
PATCH de topic	<code>.strict()</code> : rechaza claves fuera del contrato (p. ej. <code>segmentBytes</code>).
<code>query_range</code>	<code>metric</code> de un <code>enum</code> cerrado; <code>start</code> / <code>end</code> en segundos Unix; <code>step</code> / <code>window</code> duraciones Prometheus estrictas.
Login	<code>.strict()</code> : solo el token.

Un fallo de validación lanza `BadRequestException` con el detalle por propiedad, que el filtro global convierte en un `problem+json` con `issues`.

7.6 Data source de Prometheus

`PrometheusService.queryRange()` expone la historia con **degradación limpia** como comportamiento de primera clase:



Que "no hay Prometheus" responda **200 con `available: false`** y no un error es intencionado: no es un fallo, es una configuración. La SPA lo distingue y muestra un aviso honesto en lugar de una pantalla de error.

La construcción de la PromQL en servidor se detalla en el [capítulo 11](#); es también una decisión de seguridad ([capítulo 13](#)).

7.7 Servido de la SPA

En producción el BFF sirve el build estático, registrado sobre el Express subyacente **antes** del router de Nest:

1. `express.static(root, { index: false, fallthrough: true })` — entrega los ficheros reales (JS, CSS, assets). Con `fallthrough`, lo que no existe continúa.
2. **Fallback SPA** — las `GET` que no son rutas del BFF devuelven `index.html`, para que los *deep links* funcionen al recargar.

La guarda es una función explícita:

```
function isBffRoute(path: string): boolean {
  return path.startsWith('/api/') || path === '/health'
    || path === '/healthz' || path === '/readyz';
}
```

Sin ella, un `GET /api/v1/inexistente` devolvería el HTML de la SPA con status 200 en lugar de un 404 `problem+json`. Hay una prueba e2e que lo verifica.

Todo esto se activa **solo** si `WEB_DIST_PATH` apunta a un directorio con `index.html`. Si no, es un *no-op* silencioso y en desarrollo la SPA la sirve Vite. Un mismo binario, dos modos, sin ramas condicionales por entorno.

7.8 Arranque

`main.ts` es corto a propósito:

```
const app = await NestFactory.create(AppModule);
app.enableShutdownHooks(); // cierra conexiones y streams en SIGTERM/SIGINT
const config = app.get(ConfigService);
applySecurityHeaders(app, config); // antes del estático y del router: cubre todo
applyStaticHosting(app, config);
await app.listen(config.port);
```

El orden importa. Las cabeceras de seguridad se registran **primero** para que cubran también las respuestas del servicio estático, no solo las del router. Y `enableShutdownHooks()` es lo que permite que `AuthService.onModuleDestroy()` detenga el barrido de sesiones y que los SSE en vuelo se cierren limpiamente en un despliegue rodante.

Cualquier fallo de arranque —típicamente una configuración inválida— se captura en el `catch` del *bootstrap*, se registra con mensaje legible y **sale con código 1**.

8. Contrato y tipos generados

Cómo entra en la consola el contrato REST de NexusMQ, por qué está vendorizado, cómo se generan los tipos y qué hacer cuando el broker cambia. La regla de oro: **el contrato nunca se escribe a mano**.

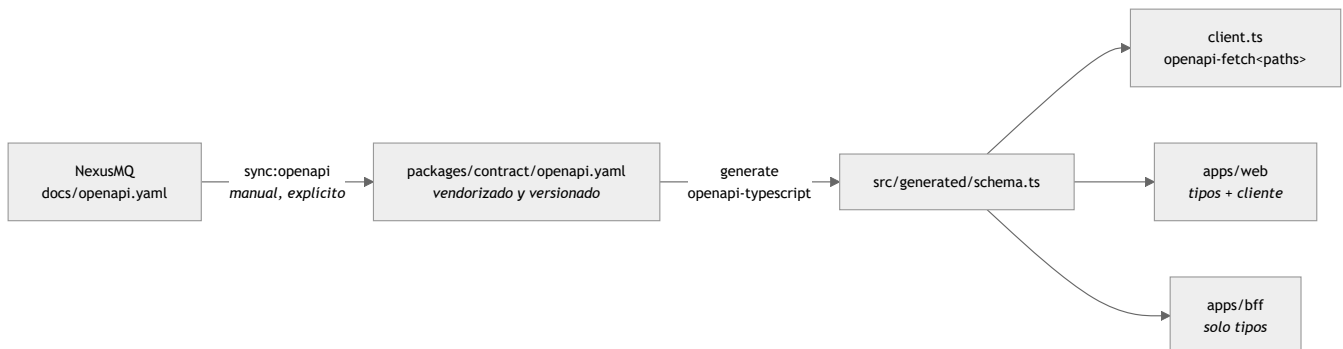
8.1 El paquete `packages/contract`

Un único paquete traduce el OpenAPI del broker a TypeScript, y todo el resto de la consola lo consume:

```
packages/contract/
├─ openapi.yaml          ← vendorizado del broker (fuente de verdad)
├─ scripts/sync-openapi.mjs ← re-descarga el YAML del repositorio del broker
├─ src/
│   ├─ generated/schema.ts ← GENERADO (en .gitignore)
│   ├─ client.ts           ← createNexusMqClient (openapi-fetch tipado)
│   └─ index.ts            ← superficie pública del paquete
```

Exporta exactamente tres cosas: el creador de cliente, sus tipos de opciones y los tipos generados (`paths` , `components` , `operations` , `webhooks`).

8.2 El pipeline de generación



Dos pasos, deliberadamente separados:

- `sync:openapi` descarga el `openapi.yaml` desde el `raw` de GitHub del broker (sobreescrible con `NEXUSMQ_OPENAPI_URL` para apuntar a un fork o a otra rama) y lo escribe en el paquete. Es **manual**: actualizar el contrato es una decisión, no un efecto colateral de un build.
- `generate` ejecuta `openapi-typescript openapi.yaml -o src/generated/schema.ts`. Es **automático**: Turborepo lo encadena antes de `build`, `typecheck` y `test`.

8.3 Por qué vendorizado

Copiar el YAML al repositorio en lugar de descargarlo en cada build compra tres propiedades:

Propiedad	Consecuencia
Build reproducible	El mismo commit produce siempre los mismos tipos, hoy y dentro de un año.
Independencia de la red	El CI no falla porque GitHub esté lento o el repositorio del broker se mueva.
Cambio de contrato visible	Actualizar el contrato produce un <i>diff</i> revisable en el historial: se ve exactamente qué cambió y cuándo.

El coste es que hay que acordarse de sincronizar. Se acepta: un contrato que cambia bajo los pies sin que nadie lo note es peor problema que uno que se actualiza tarde.

8.4 Qué cubre el contrato

El `openapi.yaml` (versión 1.0.0) define las rutas del plano de operación y sus esquemas:

Ruta	Uso en la consola
<code>GET /healthz</code> , <code>GET /readyz</code>	Sondas de vida y preparación (proxied, abiertas).
<code>GET /metrics</code>	Formato Prometheus; no lo consume la consola — lo consume Prometheus.
<code>GET /api/v1/metrics/snapshot</code>	Snapshot JSON estructurado; alimenta el Dashboard.
<code>GET/POST /api/v1/topics</code> · <code>GET/PATCH/DELETE /api/v1/topics/{name}</code>	Vista de Topics (CRUD + retención).
<code>GET /api/v1/groups</code> · <code>GET /api/v1/groups/{id}</code>	Vista de Grupos (lista + describe con lag).
<code>GET /api/v1/cluster</code>	Estado Raft, nodos y particiones; Dashboard, Cluster y Particiones.
<code>GET /api/v1/stream</code>	SSE de métricas; lo termina el BFF.

Los esquemas relevantes — `TopicSummary` , `TopicDescription` , `PartitionInfo` , `GroupSummary` , `GroupDescription` , `GroupPartitionOffset` , `NodeInfo` , `PartitionRaftInfo` , `FollowerProgress` , `ClusterInfo` , `MetricSample` , `MetricsSnapshot` , `ProblemDetail` — se usan por referencia desde ambos lados. Por ejemplo, el modelo de error de la SPA es literalmente:

```
export type ProblemDetail = components['schemas']['ProblemDetail'];
```

Autenticación: `bearerAuth` (JWT HS256), exigido solo si el nodo arrancó con `--jwt-secret` . El contrato **no define ninguna ruta que acuñe tokens** — un hecho con consecuencias arquitectónicas grandes, desarrolladas en el [capítulo 9](#).

8.5 Dónde el contrato no llega

Hay dos superficies que el OpenAPI no cubre, y ambas están documentadas como tales en el código:

Los nombres de las métricas. `MetricsSnapshot` es una **lista abierta** de `MetricSample` con `name` libre estilo Prometheus. El contrato fija la *forma*, no los *nombres*. Estos viven en el catálogo `docs/metrics.md` del broker.

Esta brecha causó el fallo más caro del proyecto (ver [capítulo 2](#) y [capítulo 11](#)); la mitigación es que los tres módulos que fijan nombres llevan un `@see` clicable a ese catálogo.

La API propia del BFF. `/api/auth/*` y `/api/history/*` son del BFF, no del broker, y por tanto no están en el OpenAPI de NexusMQ. Sus tipos se declaran a mano **a propósito**, y el código lo dice explícitamente:

```
// Nota de contrato: la forma `query_range` es del BFF (su data source de
// Prometheus), no del OpenAPI del broker, así que estos tipos se declaran a
// mano a propósito (no son tipos del contrato).
```

Distinguir "esto es del contrato" de "esto es nuestro" evita que alguien busque en el YAML un esquema que nunca estuvo ahí.

8.6 El cliente tipado

`createNexusMqClient` es una envoltura mínima de `openapi-fetch` que fija `baseUrl` como obligatoria:

```
export type NexusMqClient = Client<paths>;
export const createNexusMqClient = (o: CreateNexusMqClientOptions): NexusMqClient =>
  createClient<paths>(o);
```

En la SPA se instancia **una sola vez**, apuntando a `window.location.origin` — al BFF, mismo origen. Al ser mismo origen, la cookie `httpOnly` viaja sola y el JavaScript no la toca.

`openapi-fetch` devuelve `{ data, error, response }` en lugar de lanzar. La consola convierte eso en excepciones en un único punto (`unwrap / unwrapVoid` en `lib/problem.ts`), donde también se normaliza el error a `ProblemDetail`. Ver [capítulo 14](#).

En el BFF **no se usa el cliente**, solo los tipos: el proxy es *passthrough* con undici. Un cliente que parsea la respuesta no aportaría nada a un proxy que no la interpreta.

8.7 Procedimiento cuando el broker cambia

```
pnpm --filter @nexusmq/contract sync:openapi # trae el YAML nuevo
pnpm generate                                # regenera los tipos
pnpm typecheck                               # aquí salen las roturas
```

El `typecheck` es el detector: cada punto que usaba un campo renombrado o eliminado falla al compilar. Se arreglan, se corren los tests y se revisa el *diff* del YAML por si hay cambios semánticos que los tipos no capturan (una etiqueta nueva, un enum ampliado).

Si el cambio afecta a **nombres de métrica**, el `typecheck` **no** lo detectará —son cadenas— y hay que revisar el catálogo del broker y los tres módulos anclados a él. Está señalizado en el código y repetido aquí porque es la trampa conocida del proyecto.

`src/generated/` está en `.gitignore` a propósito: no se versiona un artefacto derivado, y así nadie puede editarlo a mano sin que el siguiente `generate` lo borre.

9. Autenticación y sesiones

El modelo de acceso de la consola: por qué el operador "pega su token", cómo se confina en servidor, cómo se descubre el modo del broker sin conocer su secreto, y qué garantiza el gate de login. Es el capítulo con más consecuencias de seguridad del documento.

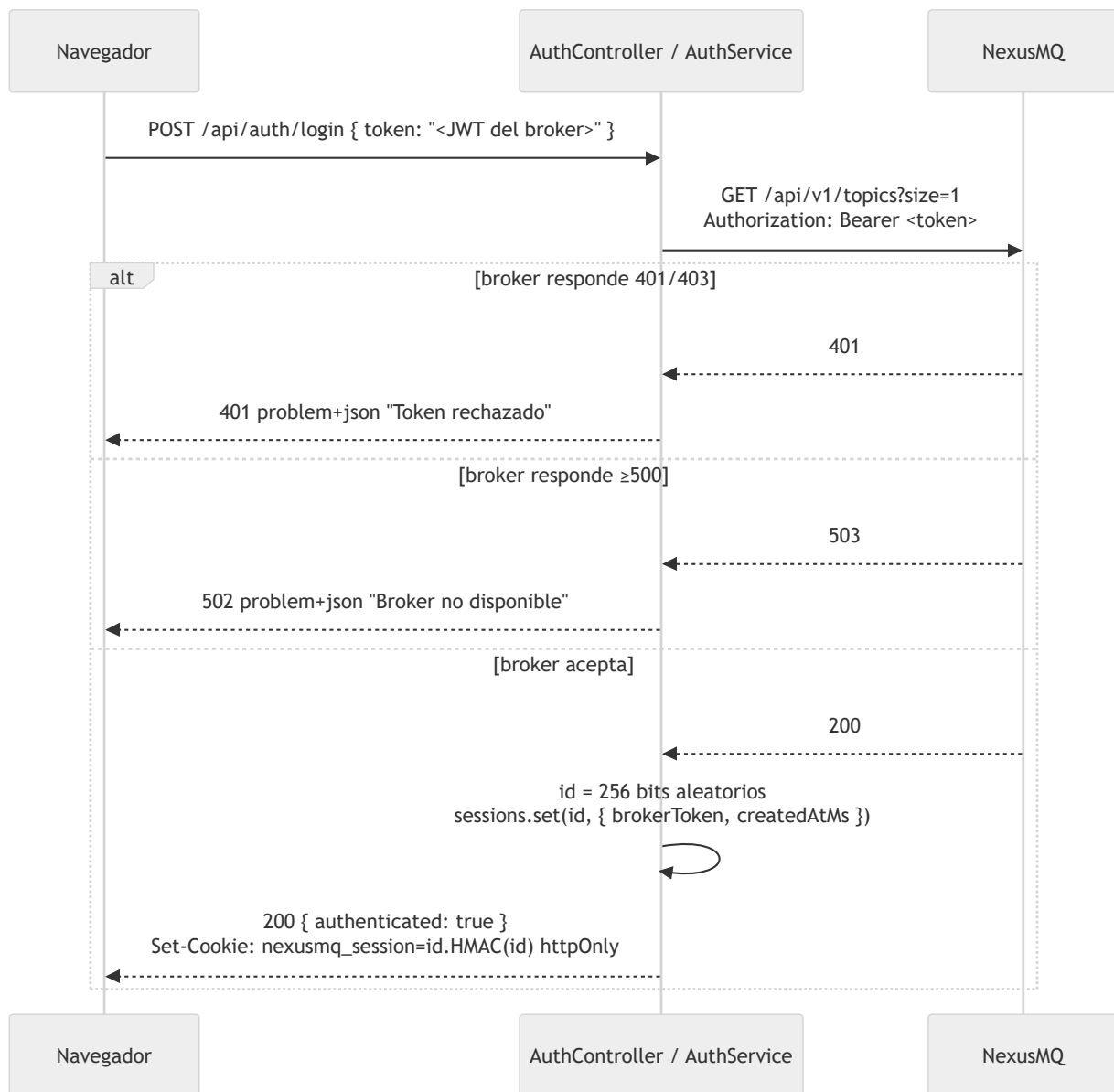
9.1 El hecho que lo condiciona todo

El contrato de NexusMQ define `bearerAuth` (JWT HS256) y lo exige **solo si el nodo arrancó con `--jwt-secret`**. Pero el OpenAPI **no define ninguna ruta que acuñe tokens**: no hay `POST /login` en el broker. Los JWT se emiten fuera, por quien custodia el secreto.

Esto descarta el flujo de login habitual —usuario y contraseña contra el backend— y obliga a un modelo distinto: **el operador pega un JWT ya emitido**. La consola no es una autoridad de identidad, y no pretende serlo.

De aquí sale la propiedad más importante del diseño: **el BFF nunca conoce el secreto HS256 del broker**. No puede firmar tokens, no puede verificarlos criptográficamente y no puede suplantar a nadie. Solo puede *usarlos y preguntarle al broker si valen*.

9.2 El flujo de login



El token se valida **usándolo de verdad** contra un endpoint protegido barato (`GET /api/v1/topics?size=1`). Si el broker lo acepta, vale; si no, no. No hay verificación local que pudiera divergir del criterio real del broker.

A partir de ahí, el token vive en un `Map` en memoria del BFF y **jamás** vuelve al navegador.

9.3 La cookie: una clave opaca, no un contenedor de datos

```
nexusmq_session = <id en hex de 256 bits> . <HMAC-SHA256(id, SESSION_SECRET) en base64url>
```

Atributo	Valor	Por qué
httpOnly	sí	El JavaScript de la página no puede leerla; un XSS no se lleva la sesión.
SameSite	Lax	Bloquea el envío en peticiones cross-site de escritura (CSRF).
Secure	en producción	Solo viaja por HTTPS. En desarrollo se desactiva para poder probar sobre <code>http://127.0.0.1</code> .
path	/	La SPA y la API comparten origen.
maxAge	SESSION_TTL_HOURS	Alineado con el TTL del servidor: no hay cookies que sobrevivan a su sesión.

La cookie **no transporta datos**, solo la clave. La firma HMAC no protege confidencialidad (no hay nada que ocultar en un id aleatorio); sirve para **descartar cookies manipuladas antes de tocar el almacén**, y se verifica en **tiempo constante** con `timingSafeEqual`.

9.4 Detección del modo del broker: sondeo, no configuración

El BFF necesita saber si el broker exige auth, pero no puede deducirlo de su configuración (no la conoce) ni del contrato (dice "depende"). Lo **sondea**:

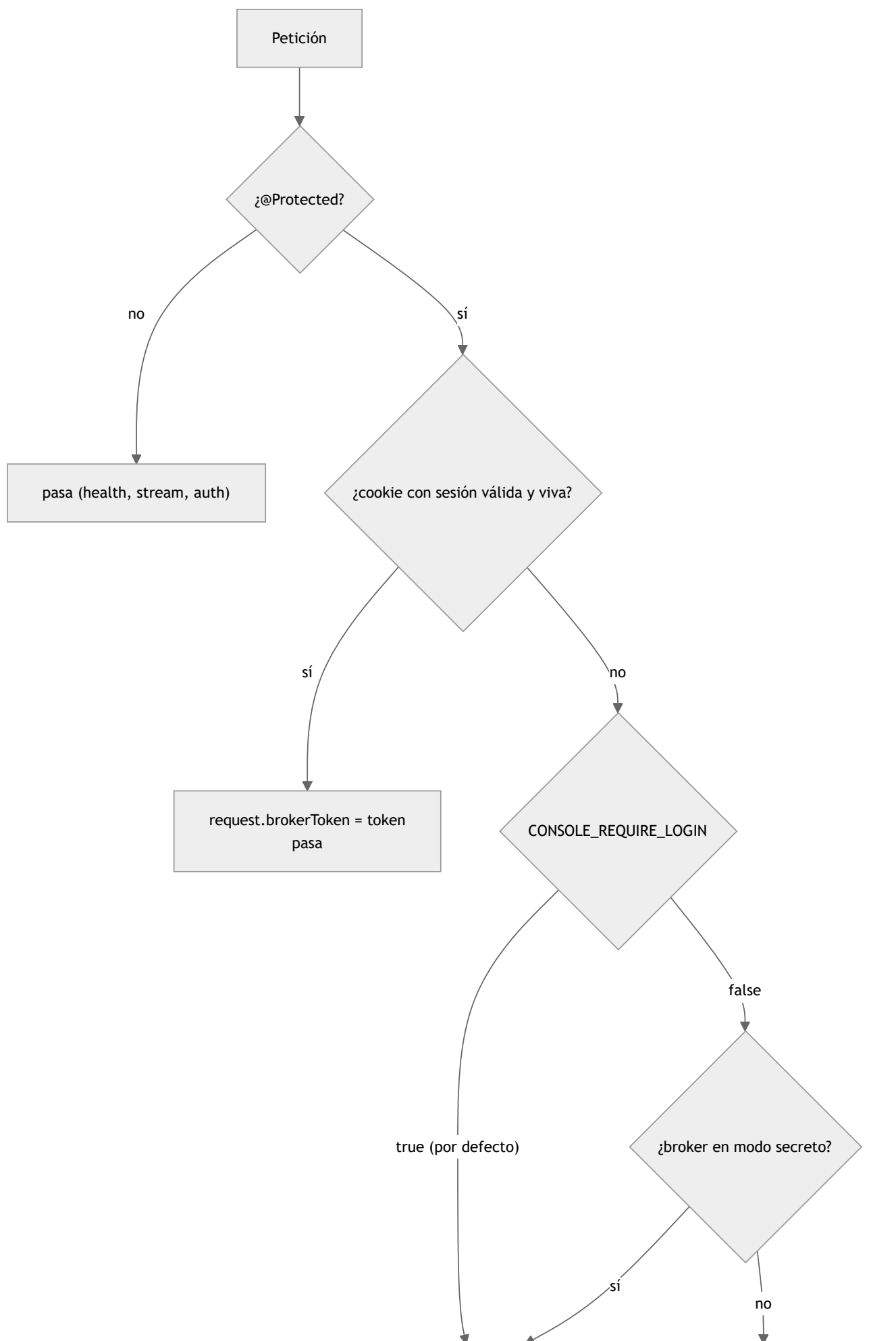
```
const result = await this.broker.forward({ method: 'GET', path: PROBE_PATH, query: { size: 1 } });
this.authRequired = result.status === 401 || result.status === 403;
```

Una petición **sin token** a una ruta protegida: `401 / 403` significa *modo secreto*; `2xx` significa *modo abierto*.

El resultado se cachea con un **TTL de 60 s**. El TTL no es una optimización: es lo que permite que, si el broker **se reinicia en otro modo**, la consola lo re-detecte sin reiniciar el BFF. Un valor cacheado para siempre habría sido un bug latente esperando a un despliegue.

9.5 El guard: cuatro caminos

`SessionAuthGuard` es global (`APP_GUARD`) pero solo actúa donde hay `@Protected()`:



El token confinado se entrega al controller mediante el decorador de parámetro `@BrokerToken()`, que lo lee de la petición. El controller nunca sabe de dónde salió.

9.6 El gate de login (`CONSOLE_REQUIRE_LOGIN`)

Por defecto `true`, **en todos los entornos**. Con él activo, las rutas `@Protected` exigen sesión **siempre**, aunque el broker esté en modo abierto.

La alternativa —espejar el modo del broker— parecía elegante y era un error: un broker en modo abierto convertía la consola en un panel de operación **sin ninguna autenticación**, accesible a cualquiera que alcanzara su URL. Que el broker no exija credenciales no significa que publicar su estado y sus mutaciones deba ser libre. Son dos decisiones distintas, y ahora se toman por separado.

`CONSOLE_REQUIRE_LOGIN=false` restaura el comportamiento anterior; existe para el desarrollo local y para despliegues donde otra capa ya autentica.

El mecanismo de login **no cambia** con el gate: sigue siendo el token del broker confinado en servidor. No se introduce una credencial de consola nueva, porque eso crearía una segunda autoridad de identidad que habría que custodiar.

9.7 Ciclo de vida de la sesión: TTL y purga

Una sesión tiene `createdAtMs` y vive `SESSION_TTL_HOURS` (8 h por defecto). La caducidad se aplica por dos vías complementarias:

- **Perezosa**, en `resolveToken()`: una sesión vencida se **borra del Map** y se trata como inexistente, sin esperar a nada. El operador ve un 401 y vuelve a entrar.
- **Periódica**, con un `setInterval` arrancado en `onModuleInit()` a cadencia `min(TTL, 10 min)`. Purga las vencidas aunque nadie las consulte, para que el `Map` no crezca con sesiones abandonadas.

El temporizador lleva `unref()` —no mantiene vivo el proceso por sí solo— y se limpia en `onModuleDestroy()`, que se invoca gracias al `enableShutdownHooks()` del arranque.

9.8 Endpoints de sesión

Endpoint	Devuelve	Nota
POST <code>/api/auth/login</code>	<code>{ authenticated: true }</code> + Set-Cookie	El cuerpo se valida con <code>.strict()</code> : solo el token.
POST <code>/api/auth/logout</code>	<code>{ authenticated: false }</code> + <code>clearCookie</code>	Idempotente: sin sesión, no falla.
GET <code>/api/auth/session</code>	<code>{ authenticated: boolean }</code>	Lo único que el navegador necesita saber.

Ninguno devuelve el token, ni su longitud, ni sus *claims*, ni el destino del broker.

9.9 El lado del cliente

La SPA deriva un **estado de acceso** de tres valores, combinando la sesión con el comportamiento observado del BFF:

```
async function resolveAccess(): Promise<AccessState> {
  const session = await fetchSession();
  if (session.authenticated) return 'authenticated';
  return probeAccess();          // sondea una ruta protegida: 401 ⇒ 'locked'
}
```

`probeAccess` es lo que hace que la SPA **se adapte sola** al gate: si el BFF exige login, la ruta protegida responde 401 y el estado pasa a `locked`, sin que el cliente tenga que conocer el valor de `CONSOLE_REQUIRE_LOGIN`. La configuración del servidor no se filtra al navegador; se **deduce de su comportamiento**.

Ese estado alimenta el guard `RequireAuth`, el indicador de conexión de la topbar y la tarjeta de sesión de Ajustes, desde la que también se puede cerrar sesión.

9.10 Qué está verificado

La no-fuga del token no se afirma, se prueba. Las pruebas e2e del BFF comprueban, sobre respuestas reales, que el JWT del broker **no aparece** en el cuerpo, ni en las cabeceras, ni en la cookie, en ninguno de los caminos de login, proxy y sesión. Además se cubren:

- 401 sin sesión en ruta protegida (modo secreto);
- login con token inválido → 401; con token válido → sesión y proxy funcionando;
- modo abierto con el gate desactivado → paso sin login;
- gate activo sobre broker abierto → 401 en `topics` y en `metrics/snapshot`, y 200 tras login;
- sesión fresca resuelve; sesión vencida da 401 y **sale del Map**;
- el sondeo del modo cachea dentro del TTL y **re-sondea** a los 61 s.

10. Tiempo real: terminación de SSE

Cómo llega al navegador el flujo en vivo del broker: por qué SSE y no WebSocket, qué significa "terminar" un stream, cómo se reconecta sin tumbar al cliente, cómo se acota la memoria por conexión y qué pasa cuando el propio plano SSE deja de estar disponible.

10.1 SSE, no WebSocket

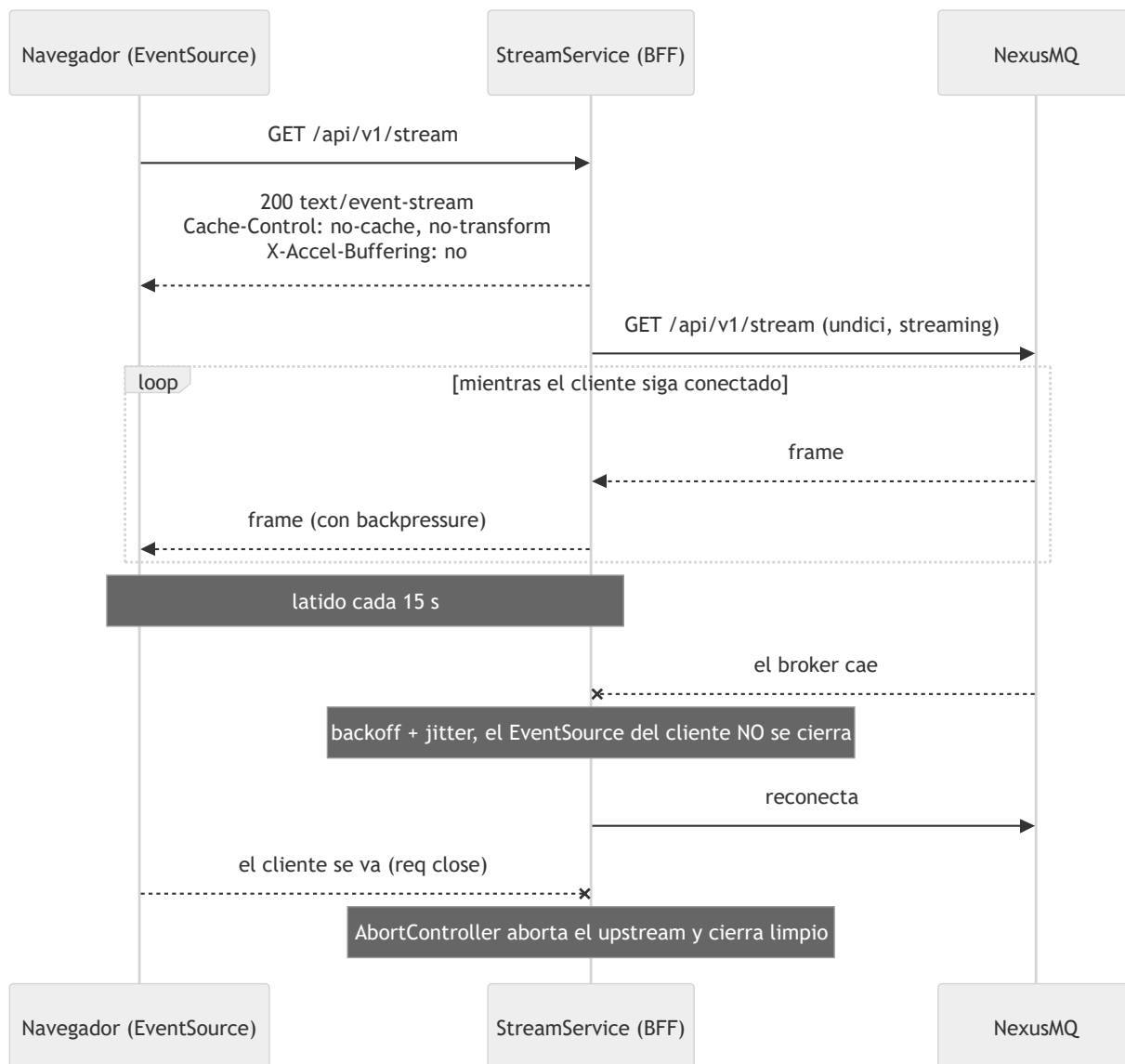
El broker expone `GET /api/v1/stream` como `text/event-stream`. La elección no es de la consola, pero es la correcta para este caso:

- el flujo es **unidireccional** (servidor → cliente); no hay nada que el navegador quiera enviar por ese canal;
- SSE va **sobre HTTP normal**, así que hereda proxies, TLS, cabeceras y cookies sin infraestructura adicional;
- `EventSource` es una API del navegador con reconexión automática de serie.

Un WebSocket habría añadido un protocolo, un *handshake* de upgrade y gestión de estado bidireccional para no usar ninguna de las dos direcciones extra.

10.2 Qué significa "terminar" el SSE

El navegador **no** abre un `EventSource` contra el broker. Abre uno contra el BFF, y es el BFF quien mantiene su propia conexión al broker y **reemite** los frames.



Esta indirección compra tres cosas que un `EventSource` directo al broker no da:

1. **Mismo origen** — sin CORS, con la cookie de sesión y bajo la CSP `connect-src 'self'`.
2. **Estabilidad frente a caídas del broker** — el cliente no ve el corte.
3. **Un solo punto de control** — *timeouts*, backpressure y cierre ordenado en servidor.

10.3 Cabeceras y latido

```

response.setHeader('Content-Type', 'text/event-stream');
response.setHeader('Cache-Control', 'no-cache, no-transform');
response.setHeader('Connection', 'keep-alive');
response.setHeader('X-Accel-Buffering', 'no');
response.flushHeaders();

```

`no-transform` y `X-Accel-Buffering: no` existen por los proxies inversos: nginx bufferiza respuestas por defecto y convertiría un flujo en tiempo real en ráfagas cada varios kilobytes.

Cada **15 s** se emite un comentario SSE (: keep-alive). Es invisible para la aplicación — `EventSource` ignora los comentarios— y mantiene viva la conexión a través de balanceadores con *idle timeout*.

10.4 Reconexión con backoff y jitter

El bucle de reenvío reintenta mientras el cliente siga conectado:

```
while (!clientSignal.aborted) {
  try {
    await this.streamOnce(response, clientSignal);
    attempt = 0; // cierre limpio del broker: reconecta desde el suelo
  } catch { /* se registra y se reintenta */ }
  if (clientSignal.aborted || !(await sleep(backoffDelayMs(attempt++, BACKOFF), clientSignal))) break;
}
```

Parámetro	Valor	Razón
Retardo inicial	500 ms	Suficiente para no martillear en un reinicio rápido.
Retardo máximo	15 s	Techo: un broker caído no se sondea cada segundo indefinidamente.
Jitter	sí	Evita que N consolas reconecten en fase y produzcan una avalancha sincronizada.
Reseteo	tras cierre limpio	Un cierre normal del broker no debe heredar el backoff de un fallo anterior.

Lo esencial: **la respuesta al navegador nunca se cierra durante la reconexión**. El `EventSource` del cliente no se entera de que el *upstream* se cayó y volvió.

10.5 Dos timeouts distintos

Un mismo `watchdog` cubre dos fases con semánticas diferentes:

- **Conexión (10 s)** — tope para *establecer* el upstream. Se limpia al recibir las cabeceras.
- **Inactividad (30 s)** — se **rearma con cada chunk**. Si el broker no emite nada en ese tiempo, se aborta y se reconecta.

El segundo es el que detecta conexiones **medio abiertas**: un socket que TCP considera vivo pero por el que nunca llegará nada más. Sin él, un stream muerto parecería un stream tranquilo.

10.6 Backpressure acotado

Es la parte menos visible y la más importante para la estabilidad.

Reemitir cada chunk con `response.write()` **sin mirar su valor de retorno** deja que Node acumule sin límite en el buffer del socket cuando el cliente es más lento que la fuente. La memoria por conexión crece sin cota: un operador con una pestaña en segundo plano y una red mala puede tumbar el proceso.

La solución respeta el contrato de los streams de Node:

```

if (response.write(chunk)) return Promise.resolve(); // cupo: seguimos
return new Promise((resolve) => { // no cupo: esperamos a 'drain'
  response.once('drain', settle);
  response.once('close', settle);
  response.once('error', settle);
  signal.addEventListener('abort', settle, { once: true });
});

```

Y el bombeo espera esa promesa entre chunk y chunk, con lo que **la lectura del upstream se pausa** hasta que el socket del cliente drene. El backpressure se propaga hasta el broker: si el operador no puede consumir, el BFF no pide más.

Los cuatro *listeners* (`drain`, `close`, `error`, `abort`) están ahí para que la promesa **siempre** se resuelva. Esperar solo a `drain` cuelga el bucle para siempre si el cliente se desconecta con el buffer lleno.

Está verificado con seis pruebas deterministas, entre ellas la que importa: **cliente lento** $\Rightarrow$ **solo se lee un chunk del upstream**, no se acumula sin límite.

10.7 Cierre limpio

Cuando el navegador cierra la pestaña, Express emite `close` en la petición. El BFF:

1. aborta el `AbortController` del cliente, lo que aborta el `fetch` al broker;
2. rompe el bucle de reconexión;
3. limpia el latido;
4. cierra la respuesta si sigue abierta.

Hay una prueba e2e que lo comprueba desde el otro lado: al irse el cliente, el doble del broker registra **cero** conexiones upstream activas. Sin esto, cada pestaña cerrada dejaría un stream huérfano consumiendo el broker.

10.8 El lado del cliente: fallback a polling

`useLiveStream` abre el `EventSource` y expone `{ data, status, source, lastUpdatedAtMs }`. Cuando el SSE falla —error fatal (`readyState === CLOSED`) o varios reintentos transitorios seguidos— **cae a polling** del snapshot cada 4 s.

Detalle que define la calidad: **el último dato persiste** al cambiar de fuente. La UI no parpadea a vacío; solo cambia el indicador de `en vivo` a `polling`. Un frame mal formado se descarta sin tumbar la conexión; un fallo puntual del snapshot se señala y se reintenta.

Merece precisión sobre **qué cubre** este fallback. El SSE del BFF ya es resiliente a las caídas del broker (§10.4), así que un corte del broker **no** tumba este `EventSource`. Lo que el fallback cubre es que el **propio plano SSE del BFF** deje de estar disponible —proxy que corta streams largos, red del cliente que no tolera conexiones persistentes. Por eso el fallo se provoca, en las pruebas, en el salto SPA $\rightarrow$ BFF.

Todo el camino está verificado en navegador full-stack: push en vivo con el BFF reemitiendo los frames del doble del broker, caída a polling al forzar el fallo, y vuelta a vivo al restaurar.

11. Observabilidad y métricas

De dónde salen los números que muestra la consola: el snapshot en vivo del broker, cómo se derivan tasas y percentiles en el cliente, cómo se ancla la consola al catálogo real de métricas, y cómo se consulta la historia en Prometheus con una allow-list de PromQL.

11.1 Dos fuentes, dos horizontes

Fuente	Horizonte	Ruta	Vista
Snapshot del broker (SSE + polling)	Ahora — segundos	<code>/api/v1/stream</code> , <code>/api/v1/metrics/snapshot</code>	Dashboard
Prometheus (<code>query_range</code>)	Historia — 15 min a 24 h	<code>/api/history/query_range</code>	Historia

La separación es deliberada y es también un no-objetivo: **la consola no almacena series**. Para el instante consulta al broker; para el pasado, a Prometheus, que es quien tiene ese trabajo. Reimplementar una TSDB estaba fuera de alcance desde el primer día.

11.2 El catálogo real del broker

El `MetricsSnapshot` del contrato es una **lista abierta** de muestras con `name` libre. El OpenAPI fija la forma, no los nombres. Los nombres viven en el catálogo `docs/metrics.md` del broker, y son estos:

Familia	Tipo	Etiquetas	Significado
<code>nexus_broker_requests_total</code>	counter	<code>{api, protocol}</code>	Peticiones del plano de datos servidas.
<code>nexus_broker_request_errors_total</code>	counter	<code>{api, protocol}</code>	Peticiones con error de wire.
<code>nexus_broker_request_bytes_total</code>	counter	<code>{api, protocol}</code>	Bytes de payload.
<code>nexus_broker_messages_total</code>	counter	<code>{api, protocol}</code>	Records (mensajes).
<code>nexus_broker_request_duration_seconds</code>	histogram	<code>{api, protocol}</code>	Latencia de servicio.
<code>nexus_broker_connections_active</code>	gauge	<code>{plane}</code>	Conexiones de cliente activas.

`api` toma `produce` o `fetch`; `protocol`, `native` o `kafka`; `plane`, `native`, `kafka` o `admin`.

La trampa que costó una fase entera. Durante las fases 1–4 la consola asumió nombres plausibles (`nexusmq_messages_in_total` ...) y los dobles de prueba emitían exactamente esos. Todos los tests pasaban; contra el broker real, el Dashboard salió vacío. El [capítulo 2](#) cuenta la lección; aquí queda su

mitigación: los **tres** módulos que fijan nombres — `metrics-snapshot.ts` (Dashboard), `history-range.ts` (ids del cliente) y `history-metrics.ts` (constructor de PromQL en servidor)— llevan un `@see` clicable al catálogo del broker. Si el catálogo cambia, el punto a tocar está señalizado.

11.3 Filtrado por etiqueta y agregación

Como cada familia viene **desglosada por etiquetas**, no basta con buscar por nombre: hay que filtrar, y a veces agregar. El modelo puro implementa el equivalente cliente de PromQL:

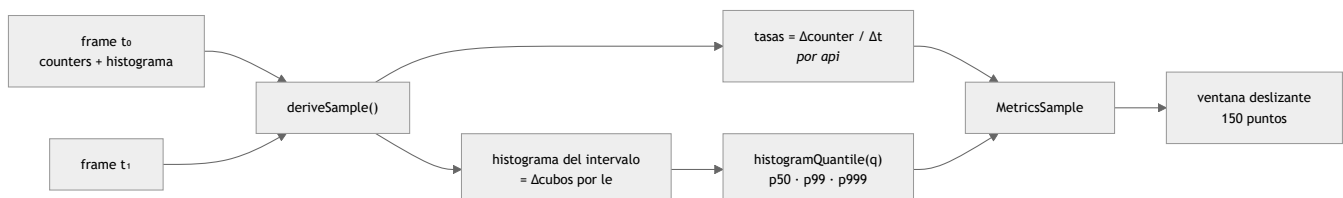
Función	Equivalente PromQL	Comportamiento
<code>findCounter(s, name, {api})</code>	<code>sum(metric{api="..."})</code>	Suma las series que casan el selector. <code>null</code> si no hay ninguna.
<code>findGauge(s, name, sel)</code>	<code>sum(gauge{...})</code>	Idem para gauges.
<code>groupGaugeByLabel(s, name, 'plane')</code>	<code>sum(gauge) by (plane)</code>	Mapa {etiqueta → suma} ordenado. <code>null</code> si no hay serie.
<code>findHistogram(s, name, {api})</code>	<code>sum(histogram{api="..."}) by (le)</code>	Suma los cubos acumulativos por <code>le</code> sobre todas las series que casan.

`findHistogram` merece explicación: sumar cubos **acumulativos** serie a serie da la distribución acumulada agregada, que es exactamente lo que hace `sum(...)` `by (le)` en Prometheus. Es lo que permite ver la latencia de `produce` sobre `native` **y** `kafka` como una sola distribución.

Todas devuelven `null` —no `0`— cuando la serie no existe. Ese `null` se propaga hasta el tile, que muestra «—». Ver [capítulo 5](#), §5.6.

11.4 Derivación en el cliente

El snapshot trae contadores acumulados y un histograma acumulado. Un dashboard necesita **tasas** y **percentiles recientes**. La derivación ocurre en el cliente, por diferencia entre el frame actual y el anterior:



Dos decisiones concretas:

Tasas por diferencia de counters, con guardas. `rate()` devuelve `null` si falta alguno de los dos valores, si $\Delta t \leq 0$ o si **el counter retrocedió** (`curr < prev`) —señal de que el broker se reinició. Un contador que baja produciría una tasa negativa absurda; es mejor un hueco honesto.

Cuantiles del histograma del intervalo, no del acumulado. `deltaHistogram()` resta los cubos del frame anterior a los del actual, y el cuantil se calcula sobre esa diferencia. Un `p99` calculado sobre el histograma desde el arranque del broker está dominado por lo que pasó hace horas y no se mueve nunca; el del

intervalo refleja **la latencia reciente**, que es la que interesa mirar. Si aún no hay intervalo (primer frame), se usa el acumulado como respaldo.

`histogramQuantile` interpola linealmente dentro del cubo, igual que Prometheus, y trata el cubo `+Inf` con cuidado: un cuantil que cae ahí devuelve el último `1e` finito en lugar de interpolar hacia infinito.

11.5 Lo que muestra el Dashboard

Seis tiles y dos gráficas en vivo:

Tile	Contenido	Degrada si...
Produce	req/s + <code>msg/s</code> · bytes/s	falta <code>messages_total</code> o <code>request_bytes_total</code> .
Fetch	req/s + <code>msg/s</code> · bytes/s	idem.
Latencia p99	p99 en ms + <code>p50</code> · p999	no hay histograma de <code>produce</code> .
Tasa de error	err/s + % sobre peticiones, con icono de estado	falta <code>request_errors_total</code> .
Conexiones activas	total + desglose por <code>plane</code>	falta <code>connections_active</code> .
Clúster	nodos + salud + término, o particiones sin líder	<code>GET /api/v1/cluster</code> no responde.

Las gráficas son **uPlot en streaming** (`setData` sin reconstruir el objeto) sobre la ventana deslizante de 150 puntos. El estado del clúster **no viaja por el SSE**, así que se sondea aparte con TanStack Query.

La tasa de error se muestra **en proporción a las peticiones**, no solo en absoluto: `50 err/s` significa cosas muy distintas con 100 o con 100 000 req/s.

11.6 Historia: allow-list de PromQL en servidor

El endpoint de historia **no acepta PromQL**. El cliente envía un **id de métrica** de una lista cerrada y el BFF construye la consulta:

```
const BUILDERS: Record<HistoryMetricId, (window: string) => string> = {
  'throughput-produce': (w) => `sum(rate(nexus_broker_requests_total{api="produce"}[${w}]))`,
  'latency-p99': (w) =>
    `histogram_quantile(0.99, sum(rate(nexus_broker_request_duration_seconds_bucket{api="produce"}[${w}]))
  errors: (w) => `sum(rate(nexus_broker_request_errors_total[${w}]))`,
  // ... 10 ids en total
};
```

Ids permitidos: `throughput-produce`, `throughput-fetch`, `latency-p50`, `latency-p99`, `latency-p999`, `errors`, `bytes-produce`, `bytes-fetch`, `messages-produce`, `messages-fetch`. Cualquier otro valor **no llega al constructor**: lo rechaza el `enum` del esquema Zod con un 400.

El único parámetro que entra en la consulta es la **ventana** (`window`), validada como duración Prometheus estricta. Nunca hay concatenación de cadenas del cliente dentro de la PromQL.

Por qué importa: un `query_range passthrough` convierte al BFF en un **motor PromQL abierto** apuntando a la Prometheus interna. Es superficie de SSRF y de DoS —una consulta con rango de un año y `step` de un segundo puede tumbar Prometheus— y una vía de exfiltración de cualquier métrica del despliegue, no solo del broker. Con la allow-list, la superficie es de diez consultas conocidas. El endpoint es además `@Protected`.

11.7 Ventanas temporales

Cuatro presets, con `step` y `window` que crecen con el rango para mantener ~120 puntos:

Preset	Rango	step	window de rate()
15 min	900 s	15 s	1 m
1 h	3 600 s	1 m	2 m
6 h	21 600 s	5 m	10 m
24 h	86 400 s	20 m	40 m

`window ≥ step` siempre: una ventana de `rate()` menor que el paso de muestreo produce series con huecos y ruido.

La respuesta `matrix` se normaliza (`NaN` de Prometheus → `null` , que uPlot dibuja como hueco) y se alinean las series sobre la **unión ordenada** de instantes, por robustez frente a series con puntos ausentes. Todo ese modelo es puro y está cubierto por pruebas.

11.8 Observabilidad del propio BFF

Endpoint	Qué reporta	Autenticación
<code>GET /health</code>	Salud del BFF : { <code>status</code> , <code>service</code> , <code>uptimeSeconds</code> }. No toca el broker.	Abierto
<code>GET /healthz</code>	Salud del broker (proxied).	Abierto
<code>GET /readyz</code>	Preparación del broker (proxied).	Abierto
<code>GET /api/history/status</code>	{ <code>available</code> } — si hay Prometheus configurado.	Abierto

`/health` no consulta el broker a propósito: un *liveness probe* debe responder si **este** proceso está vivo. Si dependiera del broker, una caída del broker provocaría que el orquestador reiniciara la consola en bucle, sin arreglar nada.

Los logs usan el `Logger` de Nest con contexto por clase y **nunca** registran el token del operador ni el contenido de la cookie.

12. Visualización

El sistema de color y los cuatro motores gráficos de la consola: qué hace cada uno, por qué conviven, cómo comparten una única fuente de color, y cómo se construyeron el Dashboard en vivo y la topología 3D del clúster.

12.1 Un sistema de color, no una paleta

Todo el color de la consola sale de `apps/web/src/styles/tokens.css`: una instancia validada de una paleta de referencia, organizada por **roles semánticos**, no por tonos.

Grupo	Tokens	Regla
Superficies	<code>--page</code> < <code>--surface</code> < <code>--elevated</code> , <code>--muted</code>	Luminancia ascendente; la jerarquía se lee sin bordes.
Tinta	<code>--foreground</code> , <code>--muted-foreground</code> , <code>--faint-foreground</code>	<code>faint</code> solo para texto no esencial (ejes, etiquetas).
Líneas	<code>--border</code> , <code>--border-strong</code> , <code>--grid</code> , <code>--axis</code>	Rejilla y eje son recesivos por definición.
Estado	<code>--success</code> , <code>--warning</code> , <code>--serious</code> , <code>--critical</code>	Nunca se tematizan : rojo es rojo en claro y en oscuro.
Categórica	<code>--series-1</code> ... <code>--series-8</code>	Orden fijo , nunca cíclico.

Tres reglas gobiernan su uso:

1. **Ningún componente elige color.** Consumen roles vía utilidades Tailwind (`bg-surface`, `text-critical`) mapeadas con `@theme inline`. Cambiar la paleta es tocar un fichero.
2. **Orden fijo en la categórica.** *Produce* es siempre `--series-1` y *fetch* siempre `--series-2`, en el Dashboard y en Historia. Un color que cambia de significado entre pantallas obliga a releer la leyenda cada vez.
3. **El color nunca informa solo.** Estado = color + **icono** + **etiqueta**. La tasa de error lleva un check o un triángulo; la salud del clúster, texto explícito; las series, leyenda.

El tema se conmuta reasignando variables (ver [capítulo 6](#), §6.6) y el contraste AA está verificado por una prueba Playwright sobre el DOM real, en ambos modos.

12.2 El puente a las gráficas: `useVizTokens`

ECharts, uPlot y react-three-fiber pintan sobre **canvas y WebGL**: no pueden usar `var(--series-1)`. Necesitan el hex ya resuelto.

`useVizTokens` lee los valores computados del `:root` y —esto es lo que lo hace fiable— los **re-lee cuando cambia el tema**, observando el atributo `data-theme` con un `MutationObserver` más el media query del sistema. La relectura se agenda en el siguiente `requestAnimationFrame`, cuando el estilo ya está recalculado.

Depender del orden de efectos de React habría producido el bug clásico: gráficas que se quedan con los colores del tema anterior hasta el siguiente re-render.

12.3 Cuatro motores, cuatro trabajos

No es acumulación: cada librería está donde hace algo que las otras hacen peor.

Motor	Dónde se usa	Por qué esa
uPlot	Dashboard en vivo, Historia	Series temporales a alta cadencia con coste mínimo. <code>setData()</code> actualiza sin reconstruir el objeto.
ECharts	Laboratorio <code>/lab</code>	Exploración y gráficas ricas con leyenda, tooltip y zoom de serie.
visx	Laboratorio <code>/lab</code>	Composición SVG: control total del marcado cuando el gráfico es parte del layout.
react-three-fiber	Topología 3D del clúster	Es la única forma razonable de mostrar un anillo de nodos con aristas de replicación en el espacio.

Los cuatro wrappers viven en `features/viz/` y comparten contrato: reciben los tokens ya resueltos, aplican grid y ejes recesivos, marcas de 2 px, y exponen `role="img"` con etiqueta descriptiva.

Sus diferencias de ciclo de vida están resueltas en el wrapper, no en el llamador:

- **EChart** — `init / dispose`, `ResizeObserver`, y `setOption` con `notMerge` para que un cambio de tema reasigne **todos** los colores en lugar de fusionarlos con los viejos.
- **UpPlotChart** — `uPlot` es imperativo y no reestiliza en caliente, así que el plot se **reconstruye** cuando cambian opciones o tema; el tamaño se sincroniza con `ResizeObserver`.
- **VisxAreaChart** — SVG puro, se re-renderiza y re-colorea solo; responsive por medición del contenedor.
- **ThreeClusterScene** — `Canvas` de `react-three-fiber` con fondo igual a la superficie del tema.

`ECharts` y `three.js` entran por `import()` diferido en rutas propias, para no cargar el *bundle* principal con lo que no se usa en el primer render.

12.4 El Dashboard en vivo

Dos gráficas `uPlot` alimentadas por la ventana deslizante de 150 muestras ([capítulo 11](#)):

- **Throughput (peticiones/s)** — `produce` y `fetch`, con relleno translúcido.
- **Latencia de servicio** — `p50`, `p99` y `p999` en milisegundos.

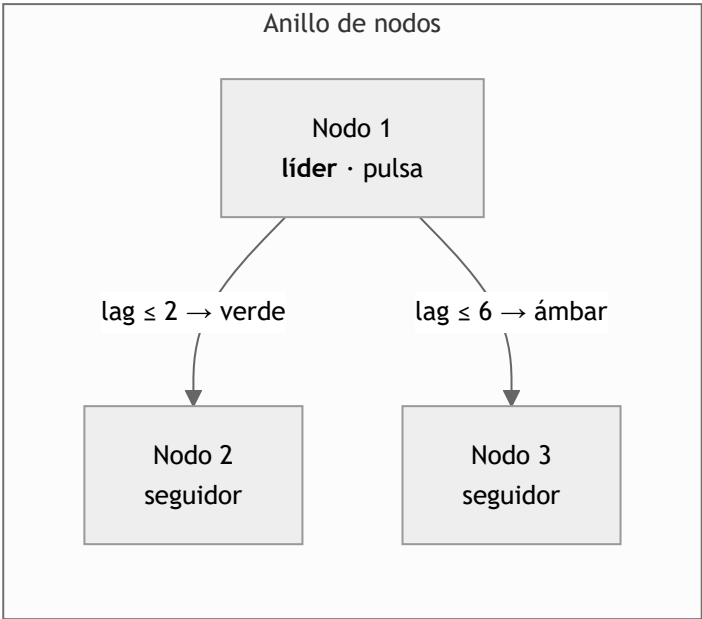
`makeLiveOptions` centraliza el estilo:

Decisión	Valor
Ejes y rejilla	<code>mutedForeground</code> y <code>--grid</code> : presentes, nunca protagonistas.
Leyenda	Obligatoria con ≥ 2 series. La identidad no puede depender del color.
Marcas	2 px, con relleno <code>rgba</code> derivado del hex de la serie.
Texto	Siempre tinta, nunca el color de la serie: un p999 escrito en rojo se lee como alarma cuando solo es una serie.
Cursor	Puntos de 6 px, sin línea horizontal (ruido en series apiladas).

12.5 La topología 3D del clúster

Es la pieza más visible y la que mejor ilustra que la visualización sirve para *entender*, no para impresionar.

Muestra los nodos del clúster como esferas en un anillo (plano XZ, ordenados por `nodeId`) y, para la **partición seleccionada**, dibuja el estado real del consenso Raft:



Señal visual	Qué codifica
Halo	El nodo local (<code>isSelf</code>).
Pulso (<i>emissive</i> oscilante)	El líder de la partición activa.
Arista líder → seguidor	Que la réplica está en el consenso.
Color de arista	Salud del <i>lag</i> : verde ≤ 2 , ámbar ≤ 6 , rojo por encima.
Grosor de arista	Magnitud del retraso.

Seleccionar otra fila en la tabla de consenso Raft cambia la partición activa y **la escena responde**: el pulso se mueve al nuevo líder y las aristas se reorientan. Hay una prueba e2e que verifica exactamente eso — seleccionar `orders.events-p2` y comprobar que el líder pasa de Nodo 1 a Nodo 2.

El color de las aristas sale de los tokens de **estado**, no de la paleta categórica: es una señal de salud, no una identidad de serie. Y como el color no puede informar solo, la tabla de consenso a su lado muestra el mismo dato en números.

Degradación honesta también aquí: un broker de un solo nodo con `replication_factor = 1` no tiene réplicas Raft. La escena muestra entonces el nodo con su halo, sin aristas, y el texto «sin particiones replicadas». No es un error de la consola; es el estado real de ese despliegue, dicho con claridad.

12.6 Accesibilidad de las gráficas

- **Etiquetadas:** cada wrapper expone `role="img"` con `aria-label` descriptivo.
- **Leyenda siempre** con dos o más series.
- **Contraste verificado** de todo el texto de la interfaz en claro y oscuro.
- **Paleta categórica** elegida para mantener separación perceptual en ambos temas.
- **Estado con icono y texto**, no solo color, en toda la superficie de la consola.

La regla de fondo: si alguien no distingue rojo de verde, la consola tiene que seguir siendo operable. Por eso ningún dato crítico viaja solo en el canal cromático.

13. Seguridad

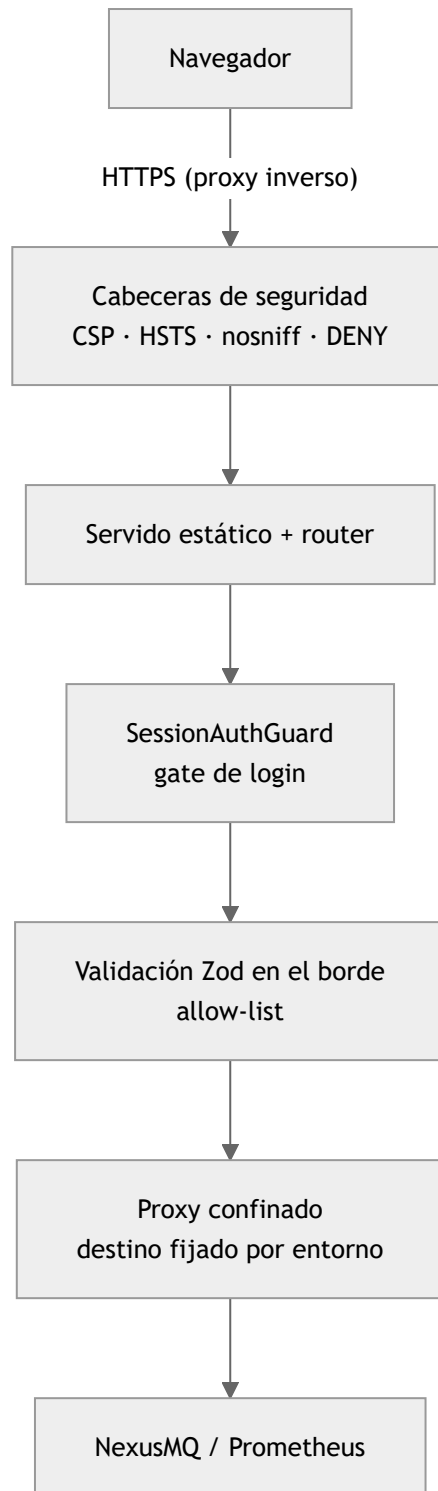
El modelo de amenazas de la consola y las defensas que lo cubren, capa por capa. Los mecanismos concretos de sesión están en el [capítulo 9](#); aquí se ve el conjunto.

13.1 Modelo de amenazas

Qué hay que proteger y de quién:

Activo	Amenaza	Defensa principal
JWT del operador	Robo por XSS, por <i>logging</i> , por respuesta filtrada	Confinado en servidor; cookie <code>httpOnly</code> ; tests de no-fuga.
El broker	Acceso directo desde el navegador; SSRF a través del BFF	El destino se fija por entorno; el navegador no conoce la URL.
Prometheus	PromQL arbitraria (SSRF, DoS, exfiltración)	Allow-list de 10 ids; la consulta se construye en servidor.
Sesión del operador	CSRF, fijación, robo por red	<code>SameSite=Lax</code> , <code>Secure</code> en producción, id de 256 bits firmado con HMAC.
La propia consola	XSS, <i>clickjacking</i> , <i>sniffing</i> de MIME	CSP anclada a <code>'self'</code> , <code>frame-ancestors 'none'</code> , <code>nosniff</code> .
Datos operativos	Exposición sin autenticación	Gate de login activo por defecto; <code>metrics/snapshot</code> protegido.

13.2 Capas de defensa



Ninguna capa es suficiente por sí sola, y esa es la idea.

13.3 Mismo origen, sin CORS

El BFF **no habilita CORS** en ningún caso. La SPA y la API comparten origen, así que:

- no se emite `Access-Control-Allow-Origin`, y el navegador bloquea cualquier lectura desde otro origen;
- la cookie funciona con `SameSite=Lax` (no hace falta `None`, que la expondría a contextos cross-site);
- `connect-src 'self'` en la CSP no necesita excepciones.

En desarrollo, Vite proxya `/api`, `/health`, `/healthz` y `/readyz` al BFF para preservar la misma propiedad en local.

13.4 Cabeceras de seguridad

Se registran sobre el Express subyacente **antes** del servidor estático y del router, de forma que cubren **toda** respuesta —HTML, assets, JSON y SSE:

Cabecera	Valor	Protege de
Content-Security-Policy	Ver §13.5	XSS, inyección de recursos.
X-Content-Type-Options	nosniff	MIME sniffing.
X-Frame-Options	DENY	Clickjacking (junto a <code>frame-ancestors 'none'</code>).
Referrer-Policy	no-referrer	Fuga de URLs internas a terceros.
Cross-Origin-Opener-Policy	same-origin	Aislamiento de contexto de navegación.
Cross-Origin-Resource-Policy	same-origin	Lectura de recursos por otros orígenes.
Origin-Agent-Cluster	?1	Aislamiento de agente.
X-DNS-Prefetch-Control	off	Resolución especulativa a terceros.
Strict-Transport-Security	max-age=15552000; includeSubDomains (solo producción)	Downgrade a HTTP.
X-Powered-By	eliminada	<i>Fingerprinting</i> del framework.

HSTS solo se emite en producción: sobre HTTP en desarrollo sería ignorada, y anunciarla desde `localhost` puede envenenar la caché HSTS del navegador para el desarrollo.

13.5 La CSP y el problema del script inline

```
default-src 'self'; base-uri 'self'; object-src 'none'; frame-ancestors 'none';
form-action 'self'; script-src 'self' 'sha256-...'; style-src 'self' 'unsafe-inline';
img-src 'self' data:; font-src 'self' data:; connect-src 'self'
```

Todo se sirve del mismo origen, así que la base es `'self'` sin excepciones de host.

Queda un problema real: el `index.html` de la SPA lleva **un script inline** —el anti-FOUC del tema, que tiene que ejecutarse antes del primer *paint* para evitar el destello de tema incorrecto. Las opciones eran `'unsafe-inline'` (que anula buena parte del valor de la CSP), un *nonce* por petición (imposible sobre un fichero estático cacheado) o un **hash**.

La consola usa el hash, **calculado en el arranque a partir del** `index.html` **realmente servido**:

```
const html = readIndexHtml(config.webDistPath);
const csp = buildCsp(html === undefined ? [] : inlineScriptHashes(html));
```

Es la diferencia entre un hash correcto y uno frágil: horneado en el código, se desincroniza en cuanto Vite regenera el script y la aplicación deja de arrancar. Calculado del artefacto desplegado, **siempre casa**.

`style-src` sí admite `'unsafe-inline'`, y conviene ser explícito sobre por qué: uPlot, ECharts y react-three-fiber fijan estilos por atributo sobre sus elementos. Es una concesión conocida y acotada al canal de estilos, que no permite ejecución de código.

13.6 Validación en el borde

Toda entrada del cliente se valida con Zod **antes** de tocar la red, con criterio de allow-list: se declara lo permitido, no lo prohibido.

Entrada	Regla	Qué evita
Paginación	<code>page ≥ 1</code> , <code>size ∈ [1, 100]</code>	Peticiones que arrastran el catálogo entero.
Nombre de topic	Obligatorio; se <code>encodeURIComponent</code> al construir la URL	Inyección de ruta.
<code>PATCH</code> de topic	<code>.strict()</code>	Envío de claves fuera del contrato.
<code>query_range</code>	<code>metric</code> de un enum cerrado; <code>start / end</code> Unix; <code>step / window</code> duraciones estrictas	PromQL arbitraria.
Login	<code>.strict()</code> , solo el token	Campos parásitos en el cuerpo.
Entorno del BFF	Esquema Zod completo, <i>fail-fast</i>	Arrancar en estado medio válido.

Los errores de validación se devuelven como `problem+json` con `issues` por propiedad: útiles para el cliente, sin filtrar detalles internos.

13.7 Anti-SSRF

Dos superficies podrían convertir al BFF en un proxy abierto, y ambas están cerradas:

El destino del broker y de Prometheus se fija por entorno. No hay ningún parámetro, cabecera ni cuerpo que permita al navegador redirigir el BFF a otro host. Es también la razón de que Ajustes **no** ofrezca cambiar de broker desde la interfaz: sería exactamente la funcionalidad que abre el agujero (ver [capítulo 19](#)).

La PromQL se construye en servidor. El cliente envía un id de una lista de diez; el BFF compone la consulta contra nombres de métrica fijos. La única parte parametrizada es la ventana, validada como duración Prometheus. La superficie pasa de "cualquier consulta contra la Prometheus interna" a "diez consultas conocidas".

13.8 Confinamiento del token

Recapitulado desde la perspectiva de seguridad:

- el JWT del broker vive en un `Map` en memoria del BFF, con TTL y purga;
- al navegador solo va `nexusmq_session = id.HMAC(id)`, `httpOnly` (inalcanzable para el JavaScript, incluso ante XSS);
- la firma se verifica en **tiempo constante** (`timingSafeEqual`);
- los endpoints de sesión devuelven exclusivamente `{ authenticated: boolean }`;
- los logs **nunca** registran el token ni la cookie.

Y no es una afirmación de diseño: hay pruebas e2e que verifican, sobre respuestas reales, que el token no aparece en cuerpo, cabeceras ni cookie en ninguno de los caminos.

13.9 Gate de login por defecto

`CONSOLE_REQUIRE_LOGIN=true` en **todos** los entornos. Un broker en modo abierto no implica que su consola deba ser pública: son dos decisiones distintas y ahora se toman por separado.

En la misma línea, `GET /api/v1/metrics/snapshot` pasó a `@Protected`. Antes era alcanzable sin sesión incluso con el broker en modo secreto: una fuga de información operativa —throughput, latencias, conexiones— para cualquiera que alcanzara la URL.

13.10 Superficie de despliegue

- **Imagen no-root**: corre como el usuario `node` (uid 1000).
- **Secretos por entorno**, nunca horneados en la imagen ni versionados. `.env` está en `.gitignore`; `.env.example` documenta las claves sin valores.
- `SESSION_SECRET` **de ≥ 32 caracteres**, exigido por el esquema; el arranque falla si no.
- **TLS terminado en un proxy inverso** delante. Con `NODE_ENV=production` la cookie es `Secure`, así que servir la consola por HTTP en producción la deja sin sesión — un fallo ruidoso, que es lo que se quiere.
- **TLS hacia el broker** validado por defecto (`BROKER_TLS_REJECT_UNAUTHORIZED=true`); relajarlo es una decisión explícita, y hay `NODE_EXTRA_CA_CERTS` para el caso legítimo de una CA propia.
- **Superficie mínima**: la imagen de runtime contiene solo las dependencias de producción del BFF, su `dist` y el build de la SPA. Sin devDependencies, sin código fuente, sin `curl` (el `HEALTHCHECK` usa el `fetch` nativo de Node).

13.11 Lo que queda fuera

Honestidad sobre los límites, con su justificación:

- **Sesiones en memoria**. Una sola instancia; escalar horizontalmente exigiría un almacén compartido. Suficiente para v1 y explícito en el diseño.
- **Sin rotación de `SESSION_SECRET` en caliente**. Cambiarlo invalida todas las sesiones —que es el comportamiento correcto, pero no hay rotación gradual.
- **Sin rate limiting** en el login. La validación del token la hace el broker, que tiene su propia protección; añadir un limitador en el BFF es trabajo futuro razonable.

- **Sin auditoría de acciones.** Quién borró qué topic y cuándo no queda registrado por la consola. Es una carencia real para un entorno multi-operador.

14. Modelo de errores

Un único formato de error de punta a punta —RFC 7807— y una regla clara de quién genera qué. Es lo que permite que la consola muestre siempre algo coherente, venga el fallo de donde venga.

14.1 Un solo formato: RFC 7807

El broker emite `application/problem+json`. El BFF también. La SPA tiene, por tanto, **un solo modelo de error** para toda la aplicación, y lo toma del contrato generado:

```
export type ProblemDetail = components['schemas']['ProblemDetail'];
```

El documento tiene la forma estándar:

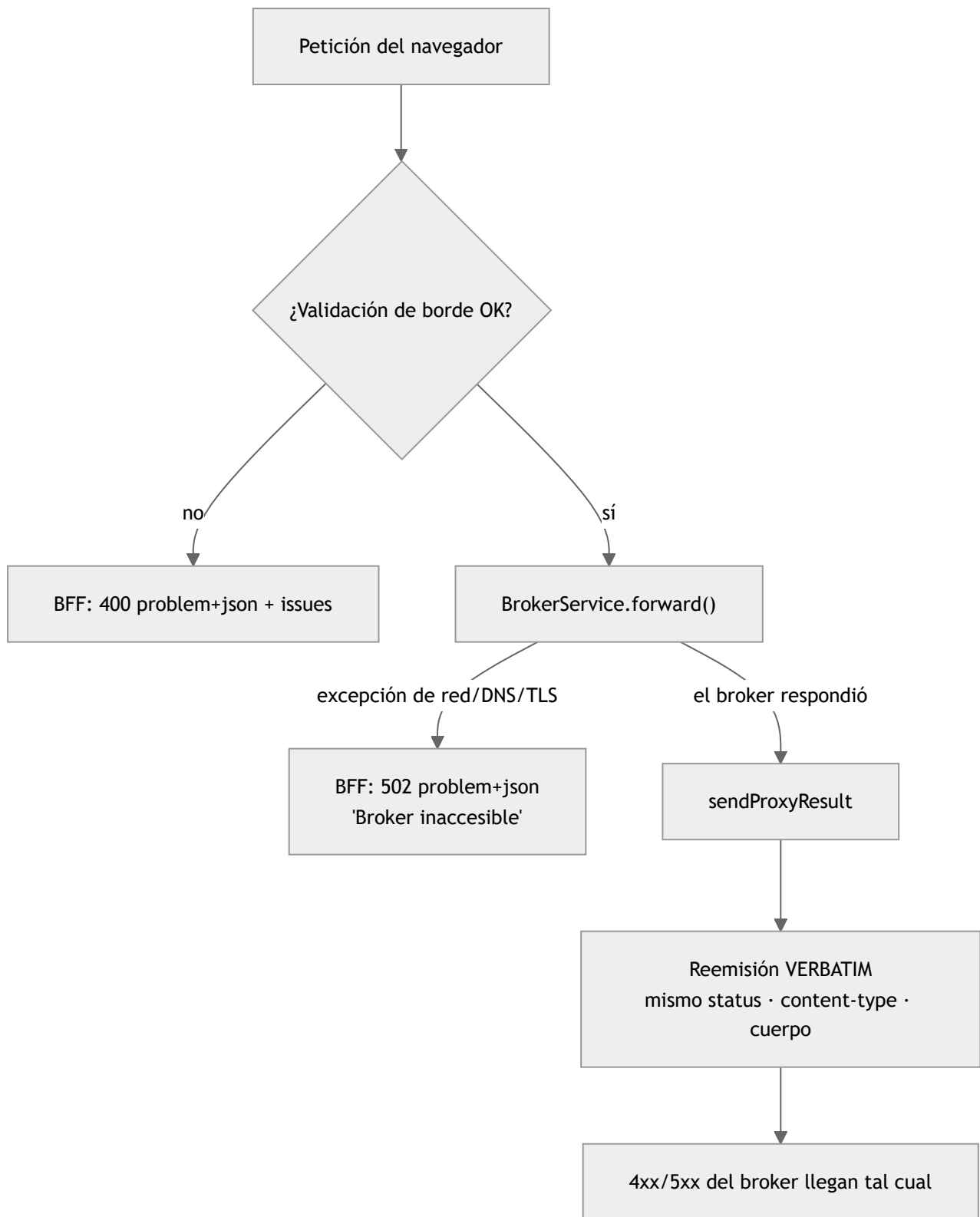
Campo	Uso
<code>type</code>	URI del tipo de problema. Por defecto <code>about:blank</code> .
<code>title</code>	Resumen corto y legible. Es lo que la UI muestra como encabezado.
<code>status</code>	Código HTTP.
<code>detail</code>	Explicación concreta de esta ocurrencia. Es lo que la UI muestra como cuerpo.
<code>instance</code>	Referencia a la ocurrencia, si aplica.
<code>issues</code>	Extensión del BFF: lista de <code>{ path, message }</code> para errores de validación.

`issues` es una extensión propia, permitida por la RFC. Sin ella, un 400 de validación diría "solicitud inválida" sin decir **qué** propiedad falla.

14.2 Quién genera qué

La regla que gobierna todo el modelo:

El BFF solo fabrica los errores que son suyos. Los del broker se reemiten intactos.



Origen	Ejemplos	Quién lo produce
Validación de entrada	<code>size=500</code> , <code>PATCH</code> con clave desconocida, <code>metric</code> fuera de la allow-list	BFF → 400 con <code>issues</code> .
Broker inaccesible	DNS, red caída, TLS, timeout	BFF → 502.
Autenticación	Sin sesión en ruta protegida, token rechazado	BFF → 401.
Prometheus	Consulta inválida → 400; caído o 5xx → 502	BFF.
Dominio del broker	Topic ya existe (409), no encontrado (404), token inválido (401)	Broker , reemitido <i>verbatim</i> .
Inesperado	Bug no capturado	BFF → 500 «Error interno», sin detalles internos.

Por qué no traducir los errores del broker: cualquier traducción crea una segunda definición del dominio que puede divergir de la primera. Si el broker devuelve un `409` explicando que el topic ya existe, ese es el mejor error posible — el BFF no tiene información adicional que aportar y sí muchas formas de estropearla.

14.3 En el BFF: el filtro global

`ProblemDetailsFilter` (`@Catch()` , registrado como `APP_FILTER`) captura **cualquier** excepción no gestionada y la traduce:

```
if (exception instanceof BrokerUnreachableError) → 502 «Broker inaccesible»
if (exception instanceof HttpException)          → status + title/detail/issues del payload
otherwise                                         → 500 «Error interno» (mensaje registrado, no expuesto)
```

Dos detalles deliberados:

- **El 500 genérico no filtra nada.** El mensaje real va al log; al cliente le llega solo «Error interno». Un *stack trace* en una respuesta es información para un atacante.
- **Las respuestas del proxy no pasan por aquí.** `sendProxyResult` escribe directamente el status y el cuerpo del broker; el filtro solo actúa sobre excepciones. Es lo que garantiza la reemisión *verbatim*.

`sendProxyResult` preserva además la cabecera `Location` cuando el broker la envía (el `201` al crear un topic), porque forma parte de la respuesta del contrato.

14.4 En la SPA: normalización en un único punto

`openapi-fetch` devuelve `{ data, error, response }` en lugar de lanzar. La conversión a excepción ocurre en un solo sitio, `lib/problem.ts`:

```
export function unwrap<T>(result: FetchResult<T>): T {
  if (result.data !== undefined) return result.data;
  throw new ProblemError(toProblem(result.error, result.response.status));
}
```

`toProblem` es la red de seguridad: si el cuerpo **no** tiene forma de `ProblemDetail` —respuesta opaca, error de red sin respuesta, HTML de un proxy intermedio— **sintetiza** uno con el status recibido. La UI siempre tiene algo coherente que mostrar; nunca un `undefined.title`.

Hay una variante `unwrapVoid` para respuestas sin cuerpo (el `204` de un `DELETE`), que solo comprueba el estado.

`ProblemError` envuelve el documento para que cualquier capa —TanStack Query, componentes— lo trate de forma uniforme, y expone `status` para las decisiones de reintento.

14.5 Reintentos: distinguir determinista de transitorio

```
retry: (failureCount, error) => {
  if (error instanceof ProblemError && error.status >= 400 && error.status < 500) return false;
  return failureCount < 2;
}
```

Un 4xx es **determinista**: el servidor ya dijo que la petición está mal, y repetirla producirá la misma respuesta. Un fallo de red o un 5xx puede ser transitorio y merece dos reintentos con *backoff*.

Reintentar un 401 solo consigue tres 401 y retrasa la redirección al login.

14.6 En la interfaz

`ProblemAlert` es el único componente de error de la consola, y aplica las reglas del sistema de diseño:

Elemento	Regla
Icono	<code>TriangleAlert</code> en color <code>critical</code> — el color nunca informa solo .
<code>title</code>	Encabezado, en tinta normal.
<code>detail</code>	Cuerpo, en tinta secundaria. Se omite si viene vacío.
<code>status</code>	«Código HTTP 409», en tinta tenue. Se omite si es 0 (error de red).
Reintento	Botón opcional, solo donde reintentar tiene sentido.
Semántica	<code>role="alert"</code> , para que los lectores de pantalla lo anuncien.

Se usa igual en el guard de acceso, en las vistas de datos y en el formulario de login. Un error de token inválido, por ejemplo, muestra el `problem+json` real del BFF —«Token rechazado» / «El broker no aceptó el token proporcionado»— y hay una prueba e2e que verifica justo eso.

14.7 Degradación frente a error

No todo lo que "no hay" es un error, y la consola distingue los dos casos:

Situación	Tratamiento	Por qué
Prometheus no configurado	<code>200 { available: false, reason }</code> + aviso honesto en la UI	Es una configuración , no un fallo. Con un error, la vista mostraría una alarma roja por algo que el operador decidió.
Métrica que el broker no emite	«—» en el tile	El dato no existe; un 0 mentiría.
Lista vacía (grupos, topics)	<i>Empty state</i> explicativo	Un resultado válido, no un error.
Prometheus configurado pero caído	<code>502 problem+json</code>	Esto sí es un fallo: algo que debería responder no responde.

La línea es clara: si el sistema está como debería y simplemente no hay dato, se degrada. Si algo que debería funcionar no funciona, se avisa.

15. Estrategia de pruebas

Qué se prueba, con qué herramienta y por qué. La tesis: **un doble solo vale si emite lo que emite el original**, y una feature con superficie visual no está hecha hasta que se ha visto funcionar en un navegador.

15.1 Los cuatro niveles

Nivel	Herramienta	Cuántas	Qué verifica
Unitario de la SPA (lógica pura)	Vitest	23 (web 22 · contract 1)	Cuantiles, tasas, ventanas, PromQL, alineado de series.
Unitario e integración del BFF	Vitest + supertest, contra dobles	82	Módulos puros (config, backpressure, CSP, allow-list) y cada endpoint sobre HTTP real.
e2e de la SPA	Playwright / Chromium sobre vite preview	7	Shell, navegación, tema, contraste AA, arsenal de viz.
e2e full-stack	Playwright sobre la topología de producción	14	SPA → BFF real → doble del broker + doble de Prometheus.

126 pruebas en total. La proporción es intencionada: la mayor parte del riesgo de esta consola está en la **frontera** —el diálogo con el broker y con el navegador—, no en funciones aisladas.

15.2 Unitario: solo lógica pura

Los tests unitarios de la SPA no montan un DOM. No hace falta, porque toda la lógica que merece prueba vive en módulos sin React:

Módulo	Qué cubre
metrics-snapshot.ts	histogramQuantile (interpolación, cubo +Inf), deltaHistogram, filtrado por etiqueta, agregación de histograma por le, groupGaugeByLabel con degradación a null.
history-range.ts	Presets de ventana, parseo de la matrix, alineado de series con huecos.

Es una consecuencia directa del principio de separar modelo puro de adaptadores ([capítulo 5](#), §5.3): si hubiera que montar un componente para probar un cuantil, la capa estaría mal puesta.

Los tests **asertan el contrato, no la implementación**. Por ejemplo, el nombre de métrica que esperan es literalmente 'nexus_broker_requests_total', y el campo de un grupo es groupId —lo que emite el broker real. Si alguien "simplifica" el nombre, el test falla.

15.3 Integración del BFF: contra dobles, sobre HTTP real

Vitest + supertest, con unplugin-swc para la metadata de decoradores de NestJS. Cada suite levanta un **doble del broker** (o de Prometheus) en un puerto efímero y ejercita el BFF de verdad por HTTP.

Cobertura, por área:

Área	Ejemplos de caso
Proxy REST	Éxito por endpoint; 404/409 del broker propagados; 400 de borde (<code>size</code> , <code>name</code> , <code>segmentBytes</code>); 502 con broker caído.
Auth	401 sin sesión; login inválido/válido; proxy con sesión; <code>session</code> ; logout; modo abierto; no-fuga del token en cuerpo, cabeceras y cookie.
Sesiones	Sesión fresca resuelve; vencida da 401 y sale del Map ; <code>purgeExpired</code> cuenta; el barrido periódico purga.
Modo del broker	Cachea dentro del TTL; re-sondea a los 61 s .
SSE	Recibe frames; reconecta manteniendo viva la conexión del cliente; cierre limpio (upstream a 0 al irse el cliente).
Backpressure	<code>write</code> true resuelve; <code>write</code> false espera a <code>drain</code> ; <code>abort</code> no cuelga; cliente lento ⇒ un solo chunk leído .
Prometheus	Con y sin Prometheus; métrica fuera de la allow-list → 400; <code>step</code> no-duración → 400; degradado → <code>available:false</code> ; inaccesible → 502; sin sesión → 401.
Cabeceras	CSP con el hash inline exacto; ausencia de <code>X-Powered-By</code> ; cabeceras presentes en respuestas reales.
Servido estático	<code>/</code> y deep links sirven <code>index.html</code> ; assets reales; <code>/health</code> sigue en JSON; <code>/api/*</code> desconocida da 404 <code>problem+json</code> ; sin <code>WEB_DIST_PATH</code> , no se sirve SPA pero la API vive.
Config	<code>validateEnv</code> rechaza entorno inválido con detalle por clave; defaults correctos.

Las pruebas de SSE y de backpressure merecen mención: son **deterministas**, no dependen de temporizadores reales ni de esperas arbitrarias. La de reconexión usa un doble que cierra tras cada frame y comprueba que hay ≥ 2 conexiones upstream mientras el cliente **no** sufre corte.

15.4 e2e de la SPA

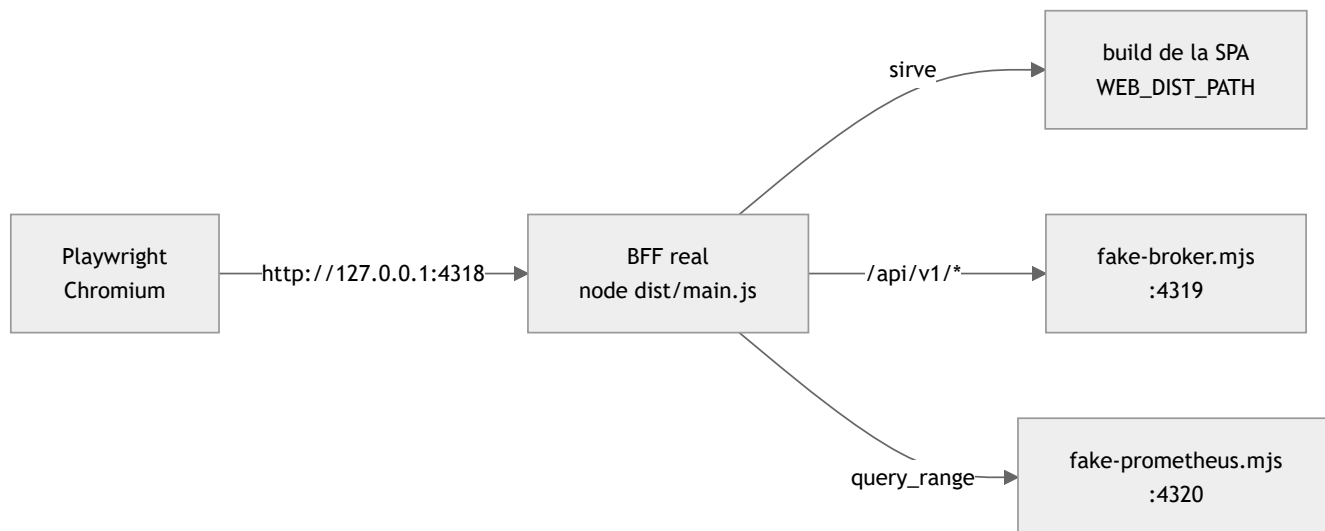
Playwright sobre el **build de producción** servido por `vite preview` — no sobre el dev server, para que se pruebe el artefacto real:

- render del shell y navegación con `aria-current` ;
- toggle de tema y persistencia al recargar;
- **contraste AA ($\geq 4.5:1$)** del texto sobre página y superficie, en claro y oscuro, medido sobre el DOM;
- 404 dentro del shell;
- el laboratorio de visualización renderiza las cuatro librerías (3 `<canvas>` incluido WebGL + SVG de visx) y respeta el tema oscuro.

Estas siete pruebas corren **sin BFF**, así que simulan la sesión con `page.route()` . Es la única suite donde se mockea la red, y es porque su objeto de prueba es la capa visual.

15.5 e2e full-stack: la topología de producción

Las 14 pruebas que más valor aportan. Playwright levanta **tres procesos**:



Es **exactamente** la topología de producción: el BFF sirviendo la SPA y proxyando, con la cookie `httpOnly` de por medio y mismo origen. No hay `page.route()` sustituyendo la API, salvo en los dos casos donde el objetivo del test **es** forzar una respuesta degradada.

`NODE_ENV` se deja sin fijar a propósito: así la cookie no es `Secure` y el navegador la acepta sobre `http://127.0.0.1`. El gate de login corre **activo**, como en producción.

Qué se verifica de punta a punta:

Suite	Verificación
<code>data</code>	Guard redirige sin sesión; deep link protegido; error RFC 7807 con token inválido; login → Topics lista los topics reales del broker ; persistencia; logout.
<code>dashboard</code>	Pasa a en vivo (SSE) , el contador de muestras avanza en < 2 s , p99 en ms, salud del clúster y estado Raft real; las dos gráficas dibujan en canvas.
<code>topics</code>	Crear → describir → PATCH de retención con efecto real («1 h» que persiste tras recargar) → borrar.
<code>groups</code>	Lista con estado → describe con miembros y lag real por partición .
<code>partitions</code>	Columna «Lag réplica»; p0 replicada con lag numérico; p3 fuera del consenso con «—».
<code>cluster</code>	Nodos, consenso Raft, topología 3D en WebGL, y el líder cambia al seleccionar otra partición.
<code>history</code>	Series desde Prometheus con leyenda; cambiar de ventana re-consulta; modo degradado con aviso honesto.
<code>live</code>	Push en vivo por SSE → caída a polling al forzar el fallo → vuelta a vivo.
<code>settings</code>	Cambio de tema, estado de conexión, y logout desde la interfaz.

El doble del broker es **stateful**: `POST / GET {name} / PATCH / DELETE` con persistencia en memoria. Sin eso, "editar la retención y ver el efecto" no sería una prueba de nada.

15.6 Los dobles emiten el contrato real

Es la lección más importante del proyecto, y ahora es una regla:

Durante las fases 1–4 los dobles emitían nombres de métrica inventados que coincidían con los que la consola esperaba. Todos los tests pasaban. Contra el broker real, el Dashboard salió vacío.

Desde la fase 5, **todos** los dobles están alineados con el contrato real del broker:

Doble	Emit
test/broker-double.ts	nexus_broker_* con {api, protocol} y {plane}; GroupDescription con groupId; ClusterInfo con {nodeId, nodes:[{nodeId, isSelf}], partitions}; páginas sin total.
test/prometheus-double.ts	nexus_broker_requests_total con api: produce.
e2e-fullstack/fake-broker.mjs	Igual, además de SSE y snapshot desde un reloj único (para que stream y snapshot sean coherentes).
e2e-fullstack/fake-prometheus.mjs	query_range con forma según la consulta.

Y los tests unitarios asertan esos mismos nombres. Un doble que confirma las suposiciones del cliente no prueba nada; solo las documenta.

15.7 Verificación en navegador como criterio de "hecho"

Ninguna feature con superficie visual se dio por terminada por compilar y pasar tests. El criterio fue **arrancarla y verla**: la vista pinta, el endpoint responde, el SSE emite, el `PATCH` tiene efecto y persiste tras recargar.

De ahí salen los detalles que ninguna prueba unitaria habría encontrado: que `uPlot` no reestiliza en caliente y hay que reconstruir el plot al cambiar de tema; que la CSP con hash inline debía calcularse del artefacto servido y no hornearse; que Turborepo filtraba el entorno de la tarea `dev` y el BFF abortaba.

15.8 Cómo se ejecuta

```
pnpm test # unitario + integración (105)
pnpm --filter @nexusmq/web test:e2e # e2e SPA (7, Chromium)
pnpm --filter @nexusmq/web test:e2e:data # e2e full-stack (14, SPA→BFF→dobles)
```

`test:e2e:data` construye antes el BFF y la SPA: prueba **artefactos**, no fuentes. Si Playwright pide navegador: `pnpm exec playwright install chromium`.

16. Puerta de calidad y CI/CD

Lo que tiene que estar verde antes de cada *push*. La puerta es **obligatoria**, se ejecuta en local y el CI la replica. Nunca se sube en rojo.

16.1 La puerta, en orden

```
pnpm install --frozen-lockfile
pnpm generate          # tipos del OpenAPI (openapi-typescript)
pnpm lint              # eslint
pnpm format:check      # prettier --check
pnpm typecheck         # tsc --noEmit en los 3 paquetes
pnpm build             # vite build + tsc
pnpm test              # unitario + integración (105)

pnpm --filter @nexusmq/web test:e2e      # e2e SPA (7, Chromium)
pnpm --filter @nexusmq/web test:e2e:data # e2e full-stack (14)
```

El orden no es arbitrario:

- **generate** **va antes que todo lo demás**. Sin los tipos del contrato, `typecheck`, `build` y `test` no tienen nada válido contra lo que compilar. Turborepo lo encadena como dependencia, pero en el CI se hace explícito para que un fallo ahí se lea en su propio paso.
- **lint** y **format:check** **van antes que typecheck**. Son los más rápidos; fallar pronto ahorra minutos.
- **Los e2e van al final**. Necesitan artefactos contruidos y levantan procesos reales.

16.2 Qué garantiza cada paso

Paso	Herramienta	Qué impide que entre
<code>generate</code>	<code>openapi-typescript</code>	Que el contrato vendorizado y los tipos diverjan.
<code>lint</code>	ESLint 9 (flat config) + <code>typescript-eslint</code> + reglas de React Hooks	Dependencias de hooks mal declaradas, <code>any</code> implícitos, importaciones desordenadas.
<code>format:check</code>	Prettier	Ruido de estilo en los <i>diffs</i> .
<code>typecheck</code>	<code>tsc --noEmit</code> en los 3 paquetes	Roturas de contrato, tipos incompatibles. Es el detector de cambios del broker.
<code>build</code>	Vite (SPA) + <code>tsc</code> (BFF y contract)	Que algo compile en desarrollo pero no en producción.
<code>test</code>	Vitest (+ supertest)	Regresiones de lógica y de endpoints.
<code>test:e2e</code>	Playwright / Chromium	Regresiones visuales, de tema y de contraste AA.
<code>test:e2e:data</code>	Playwright full-stack	Regresiones de integración real SPA → BFF → broker.

`typecheck` merece énfasis: es el mecanismo por el que un cambio del contrato del broker se manifiesta como **error de compilación** en cada punto afectado, en lugar de como un `undefined` en producción. Es la razón de que el contrato se genere y no se escriba.

16.3 ESLint y Prettier: responsabilidades separadas

- **ESLint** decide sobre **corrección**: hooks, tipos, patrones peligrosos. `eslint-config-prettier` desactiva sus reglas estéticas para que no compita con Prettier.
- **Prettier** decide sobre **formato**, y su `.prettiignore` excluye tres cosas a propósito:
 - los **artefactos generados** (`packages/contract/src/generated`, `dist`, `coverage`);
 - el `openapi.yaml` **vendorizado** — es una copia fiel del broker; reformatearlo produciría un *diff* espurio en cada sincronización;
 - los **Markdown** — la documentación tiene saltos de línea, tablas y diagramas Mermaid intencionados. Prettier formatea código y configuración, no prosa.

El paso `format:check` en el CI se añadió al cerrar el proyecto: hasta entonces la comprobación existía como script pero no como puerta, y unos veinte ficheros habían derivado. Una regla sin gate es una sugerencia.

16.4 El pipeline de CI

`.github/workflows/ci.yml`, en cada *push* a `main` y en cada *pull request*:

```
jobs:
  verify:
    name: install · lint · format · typecheck · build · test
    runs-on: ubuntu-latest
    steps:
      - Checkout
      - Instalar pnpm                # pnpm/action-setup@v4
      - Configurar Node              # node-version-file: .nvmrc (Node 22 LTS) + cache: pnpm
      - Instalar dependencias        # pnpm install --frozen-lockfile
      - Generar contrato             # pnpm generate
      - Lint                         # pnpm lint
      - Format                       # pnpm format:check
      - Typecheck                   # pnpm typecheck
      - Build                       # pnpm build
      - Test                        # pnpm test
```

Con `permissions: contents: read` (mínimo privilegio) y `concurrency` con `cancel-in-progress`, para que un *push* nuevo cancele el run anterior de la misma rama.

La versión de Node sale de `.nvmrc`, el mismo fichero que usa el desarrollo local: **una sola fuente de verdad** para la versión del runtime.

`--frozen-lockfile` es lo que convierte el CI en una verificación real: si alguien tocó un `package.json` sin actualizar el *lockfile*, el CI falla en lugar de instalar silenciosamente otra cosa.

16.5 Caché e incrementalidad

Turborepo cachea por tarea y por hash de entradas, tanto en local como en CI (con la caché de pnpm de `actions/setup-node`). En la práctica, un cambio que solo toca la SPA no vuelve a compilar ni probar el BFF.

La tarea `dev` declara `passThroughEnv` con sus nueve variables. Turborepo 2.x usa *env-mode* estricto: sin esa declaración filtra las variables no listadas, y el BFF arrancaba con el entorno vacío y abortaba en la validación de configuración. La caché no se ve afectada — `passThroughEnv` no entra en el hash.

16.6 Los e2e no están en el CI (y por qué)

El workflow ejecuta hasta `test`. Las dos suites Playwright se ejecutan **en local** antes de cada *push*.

Es una decisión consciente sobre el coste: los e2e full-stack levantan tres procesos, requieren Chromium descargado y construyen ambos artefactos. Añadirlos al workflow multiplicaría el tiempo de cada *push* a cambio de repetir una verificación que ya se hizo con el navegador delante.

Es también una **limitación conocida**: si alguien contribuyera sin ejecutar los e2e, el CI no lo detectaría. Está anotada como trabajo futuro en el [capítulo 19](#); la solución natural es un job aparte con `playwright install -with-deps chromium` y ejecución solo en `main` o bajo etiqueta.

16.7 Definición de "hecho"

Un cambio está terminado cuando:

1. cumple su criterio de aceptación;
2. pasa la puerta completa, e2e incluidos;
3. respeta las convenciones (arquitectura, naming, errores, seguridad);
4. **no deja TODO en el código** — lo pendiente se documenta, no se comenta;
5. si tiene superficie de *runtime*, **se ha arrancado y comprobado de verdad**: la vista pinta, el endpoint responde, el SSE emite.

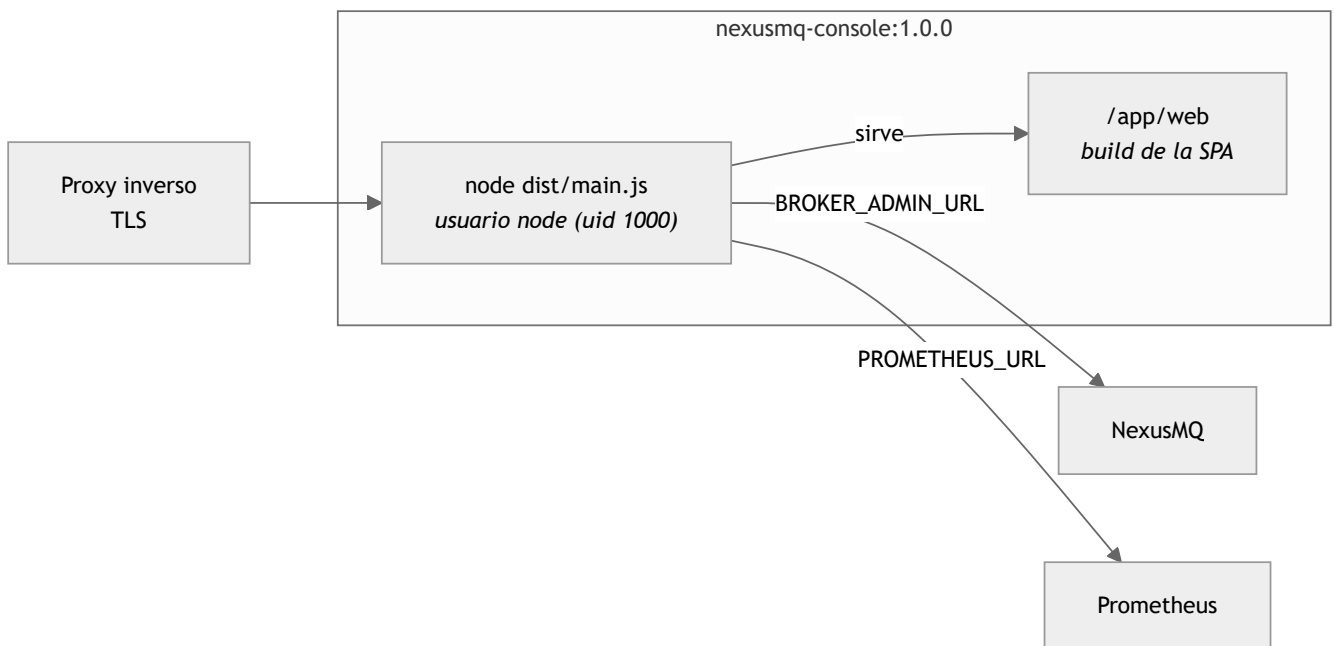
El punto 5 es el que separa "compila" de "funciona", y es donde aparecieron casi todos los problemas interesantes del proyecto.

17. Despliegue

Cómo se empaqueta y se pone en marcha la consola: una imagen Docker multi-stage, un `docker-compose` de ejemplo y las condiciones de producción (TLS, *probes*, red).

17.1 Un solo artefacto

La consola se despliega como **una sola imagen**: el BFF NestJS que sirve la SPA ya construida. No hay nginx delante sirviendo estáticos, ni dos contenedores que coordinar.



El broker y Prometheus son **externos** y se apuntan por variables de entorno. Nunca se hornean en la imagen: la misma imagen sirve para todos los entornos, y lo que cambia es la configuración (12-factor).

17.2 La imagen: multi-stage

Etapas builder (`node:22-slim` , `pnpm` vía `corepack`):

1. **Manifiestos primero** — `pnpm-lock.yaml` , `pnpm-workspace.yaml` y los cuatro `package.json` , seguidos de `pnpm install --frozen-lockfile` . Así la capa de dependencias **se cachea** y no se invalida al cambiar código.
2. **Código y build** — `pnpm generate && pnpm build` : tipos del contrato, SPA con Vite, BFF con `tsc` .
3. **Aislamiento del runtime** — `pnpm --filter @nexusmq/bff --legacy deploy --prod /out` produce un `node_modules` autocontenido con **solo** las dependencias de producción del BFF. Sin `devDependencies` y sin `@nexusmq/contract` , que en el BFF es *type-only* y desaparece al compilar. El flag `--legacy` es necesario porque el workspace no inyecta paquetes.

Etapas runtime (`node:22-slim` , mínima):

```
ENV NODE_ENV=production PORT=3000 WEB_DIST_PATH=/app/web
COPY --from=builder --chown=node:node /out/node_modules ./node_modules
COPY --from=builder --chown=node:node /out/dist ./dist
COPY --from=builder --chown=node:node /out/package.json ./package.json
COPY --from=builder --chown=node:node /repo/apps/web/dist ./web
USER node
```

Solo el artefacto: sin código fuente, sin herramientas de build, sin devDependencies. Corre como el usuario **no-root** `node` (uid 1000), que la imagen base ya trae.

17.3 HEALTHCHECK propio

```
HEALTHCHECK --interval=30s --timeout=5s --start-period=20s --retries=3 \
  CMD node -e "fetch('http://127.0.0.1:'+(process.env.PORT||3000)+'/health')\
    .then(r=>{if(!r.ok)process.exit(1)}).catch(()=>process.exit(1))"
```

Dos decisiones:

- **Comprueba que el BFF responde de verdad**, no solo que el proceso vive. Un proceso Node colgado sigue existiendo.
- **Usa el `fetch` nativo de Node 22**, no `curl`. La imagen base no lo trae, y añadirlo solo para el healthcheck ampliaría la superficie sin necesidad.

`/health` es del BFF y **no** consulta al broker, deliberadamente: si dependiera del broker, una caída del broker haría que el orquestador reiniciara la consola en bucle sin arreglar nada.

17.4 Ejecución directa

```
docker build -t nexusmq-console:1.0.0 .

docker run --rm -p 3000:3000 \
  -e BROKER_ADMIN_URL=http://broker:8080 \
  -e PROMETHEUS_URL=http://prometheus:9090 \
  -e SESSION_SECRET="$(openssl rand -hex 32)" \
  nexusmq-console:1.0.0
```

`SESSION_SECRET` es obligatorio y debe tener ≥ 32 caracteres. Si falta, el arranque **falla con mensaje claro y código 1** — no arranca en un estado degradado silencioso.

17.5 docker-compose de ejemplo

`docker-compose.yml` levanta **console + broker + prometheus**:

```
export SESSION_SECRET="$(openssl rand -hex 32)"
docker compose up --build      # consola en http://localhost:3000
```

Servicio	Notas
console	Se construye del Dockerfile . SESSION_SECRET con sintaxis \${VAR:?mensaje} : compose falla si no está definida. Healthcheck y restart: unless-stopped .
broker	Placeholder. Sustituye su image: por la del broker NexusMQ real. Expone :8080 (admin) y :9091 (métricas).
prometheus	prom/prometheus:v3.1.0 , con deploy/prometheus.yml montado en solo lectura y un volumen para los datos.

deploy/prometheus.yml raspa broker:9091/metrics cada 15 s. Ese es el camino completo: broker → Prometheus → BFF (query_range) → vista de Historia.

Solo el puerto de la consola se publica al host; broker y Prometheus quedan en la red interna de compose. El de Prometheus está comentado para poder inspeccionarlo durante el desarrollo.

17.6 Producción y TLS

La consola habla **HTTP en claro** dentro de su red y **termina TLS en un proxy inverso** delante (nginx, Traefik, ingress de Kubernetes).

Con NODE_ENV=production —que la imagen ya fija— la cookie de sesión es Secure y se emite HSTS. La consecuencia práctica es directa: **servirla por HTTP en producción la deja sin sesión**. Es un fallo ruidoso y buscado, preferible a una cookie de sesión viajando en claro.

Si el proxy hace *buffering*, hay que desactivarlo para /api/v1/stream . El BFF ya envía X-Accel-Buffering: no y Cache-Control: no-transform , que nginx respeta.

Para las *probes* del orquestador:

Probe	Endpoint	Por qué
Liveness	GET /health	Salud del BFF. No depende del broker.
Readiness	GET /readyz	Preparación del broker: si el broker no está listo, la consola no tiene nada que servir.

17.7 Estado de verificación

Honestidad sobre qué se probó y cómo: **Docker no estaba instalado en el entorno de desarrollo**, así que docker build y docker compose up no llegaron a ejecutarse aquí. El runtime de la imagen se reprodujo **sin el demonio**, y eso sí quedó verificado:

- pnpm --filter @nexusmq/bff --legacy deploy --prod produce el bundle correcto (dependencias de runtime exactas, contrato ausente por ser type-only);
- arrancar node dist/main.js con el entorno de producción **sirve la SPA** (/ , <title> , assets), el **deep link** /history (fallback SPA → 200), /health y /api/history/status ;
- el **comando del HEALTHCHECK** devuelve exit 0 sano y exit 1 con el servicio caído;

- las cabeceras de seguridad aparecen en respuestas reales, con el **hash CSP exacto** del index servido;
- el YAML de compose y el de Prometheus validan.

Queda pendiente ejecutar `docker compose up` en una máquina con Docker. Está anotado en el [capítulo 19](#).

18. Configuración y operación

Todas las variables de entorno del BFF, cómo se validan, y las tareas habituales de operación: arrancar en local, diagnosticar, y qué mirar cuando algo no va.

18.1 Configuración por entorno, validada al arrancar

La consola es **12-factor**: toda su configuración entra por variables de entorno, y el mismo artefacto sirve para todos los despliegues. No hay ficheros de configuración por entorno, ni builds distintos para desarrollo y producción.

El esquema Zod de `config.schema.ts` es una **allow-list**: solo estas claves, con su tipo, su rango y su valor por defecto. `validateEnv()` es una función pura (y por tanto testeable) que se invoca desde el constructor de `ConfigService`; un fallo lanza `ConfigValidationError` con el detalle **por clave** y `main.ts` sale con código 1.

Configuración de entorno inválida:

- `BROKER_ADMIN_URL`: requerida: URL del plano admin del broker
- `SESSION_SECRET`: debe tener al menos 32 caracteres

Corrige las variables y reinicia el BFF.

Nunca arranca a medias. Es la aplicación del principio *fail-fast* ([capítulo 5, §5.7](#)).

18.2 Variables de entorno

Plantilla completa en `.env.example`.

Variable	Obligatoria	Por defecto	Descripción
SESSION_SECRET	Sí	—	Firma la cookie de sesión <code>httpOnly</code> . ≥ 32 caracteres (<code>openssl rand -hex 32</code>).
BROKER_ADMIN_URL	Sí	—	URL del plano de operación (admin) del broker. Validada como URL.
PROMETHEUS_URL	No	—	Data source de historia. Sin él, la vista de Historia degrada limpio .
SESSION_TTL_HOURS	No	8	TTL de la sesión de operador (máx. 720). Vencida, se purga y se exige re-login.
CONSOLE_REQUIRE_LOGIN	No	true	Exige sesión siempre , aunque el broker esté en modo abierto. <code>false</code> espeja el modo del broker.
PORT	No	3000	Puerto de escucha.
NODE_ENV	No	development (production en la imagen)	<code>production</code> ⇒ cookie <code>Secure</code> + HSTS + servido estático.
BROKER_TLS_REJECT_UNAUTHORIZED	No	true	Validar el certificado TLS del broker al proxyar.
NODE_EXTRA_CA_CERTS	No	—	Ruta a un PEM con CAs adicionales (lo lee Node de forma nativa).
WEB_DIST_PATH	No	/app/web (en la imagen)	Directorio del build de la SPA. Si no apunta a uno con <code>index.html</code> , no se sirve SPA.

Notas de operación por variable

- `SESSION_SECRET` — cambiarlo **invalida todas las sesiones** en curso. Es el comportamiento correcto para una rotación, pero implica que todos los operadores vuelven a entrar. No hay rotación gradual.
- `CONSOLE_REQUIRE_LOGIN` — dejarlo en `true` salvo que otra capa (un proxy con SSO, una VPN) ya autentique. Ponerlo en `false` con el broker en modo abierto deja la consola **pública**.
- `BROKER_TLS_REJECT_UNAUTHORIZED=false` — solo para desarrollo con certificados autofirmados. En producción, la respuesta correcta es `NODE_EXTRA_CA_CERTS` con la CA propia.
- `SESSION_TTL_HOURS` — el `maxAge` de la cookie se alinea automáticamente, así que no hay cookies que sobrevivan a su sesión de servidor.

18.3 Puesta en marcha en local

```
pnpm install
pnpm --filter @nexusmq/contract generate

export BROKER_ADMIN_URL=http://localhost:8080
export SESSION_SECRET="$(openssl rand -hex 32)"
pnpm dev          # BFF (:3000) + SPA con HMR (:5173)
```

La SPA se abre en `http://localhost:5173`; Vite proxya `/api`, `/health`, `/healthz` y `/readyz` al BFF, de modo que el navegador ve **un solo origen** también en desarrollo y la cookie se comporta igual que en producción.

Para probar contra un broker real en modo secreto: arráncalo con `--jwt-secret`, emite un JWT HS256 con ese secreto y pégalo en el login de la consola.

Nota de plataforma. Si el broker corre en WSL y la consola en Windows (o viceversa), `BROKER_ADMIN_URL` debe apuntar a la IP alcanzable entre ambos, no a `localhost`. Y para probar sesión sobre `http://`, deja `NODE_ENV` sin fijar (o en `development`) para que la cookie no sea `Secure`.

18.4 Scripts disponibles

Comando	Efecto
<code>pnpm dev</code>	BFF + SPA en desarrollo.
<code>pnpm build</code>	Build de producción de todo el monorepo.
<code>pnpm test</code>	Unitario + integración.
<code>pnpm lint</code> · <code>pnpm typecheck</code>	Calidad estática.
<code>pnpm format</code> · <code>pnpm format:check</code>	Formato (escribe / comprueba).
<code>pnpm --filter @nexusmq/contract sync:openapi</code>	Re-descarga el OpenAPI del broker.
<code>pnpm --filter @nexusmq/contract generate</code>	Regenera tipos + cliente.
<code>pnpm --filter @nexusmq/web test:e2e</code>	e2e de la SPA.
<code>pnpm --filter @nexusmq/web test:e2e:data</code>	e2e full-stack.

18.5 Diagnóstico

Qué mirar primero según el síntoma:

Síntoma	Comprobación	Causa habitual
El BFF no arranca	Su log: dice qué clave falla	Falta <code>BROKER_ADMIN_URL</code> o <code>SESSION_SECRET</code> , o <code>SESSION_SECRET</code> es corto.
Todo da 502	<code>GET /healthz</code> (proxied al broker)	Broker inalcanzable: DNS, red, <code>BROKER_ADMIN_URL</code> mal, o TLS rechazado.
Redirige al login en bucle	<code>GET /api/auth/session</code>	La cookie no se guarda: <code>NODE_ENV=production</code> sobre HTTP ⇒ cookie <code>Secure</code> descartada.
401 tras un rato	TTL de sesión	<code>SESSION_TTL_HOURS</code> vencido; el BFF purga y exige re-login.
Historia vacía con aviso	<code>GET /api/history/status</code>	<code>PROMETHEUS_URL</code> no configurada (degradación, no fallo).
Historia da 502	Prometheus directamente	Prometheus configurado pero caído o inalcanzable.
Dashboard con todo «—»	Snapshot: nombres de las series	El broker emite otros nombres → revisar su catálogo de métricas (capítulo 11).
El SSE no llega	Cabeceras de la respuesta a <code>/api/v1/stream</code>	Proxy inverso bufferizando: falta desactivarlo para esa ruta.
Grupos vacíos / Raft sin aristas	Estado real del broker	No es un bug: single-node con RF=1 no tiene réplicas, y sin consumidores no hay grupos.

Los logs del BFF usan el `Logger` de Nest con contexto por clase (`AuthService`, `BrokerService`, `StreamService`, `SecurityHeaders`, `StaticHosting`). **Nunca** registran el token del operador ni el contenido de la cookie.

Mensajes de arranque útiles para verificar un despliegue:

```
[SecurityHeaders] Cabeceras de seguridad activas (CSP con 1 hash(es) inline).
[StaticHosting]   Sirviendo la SPA desde /app/web
[Bootstrap]      BFF escuchando en http://localhost:3000
```

Si falta el de `StaticHosting`, `WEB_DIST_PATH` no apunta a un directorio con `index.html` y la consola servirá la API pero no la interfaz. Si la CSP dice «0 hash(es)», tampoco encontró el index.

18.6 Operación habitual

Iniciar sesión. El operador pega un JWT del broker ya emitido. El BFF lo valida contra el broker y lo confina en servidor; al navegador solo va la cookie. Ver [capítulo 9](#).

Cambiar de broker. Se hace **por entorno**, no desde la interfaz: modifica `BROKER_ADMIN_URL` y reinicia. La interfaz no lo permite a propósito ([capítulo 13](#), §13.7).

Actualizar el contrato tras un cambio del broker.

```
pnpm --filter @nexusmq/contract sync:openapi
pnpm generate && pnpm typecheck
```

Y si cambiaron **nombres de métrica**, revisar el catálogo del broker: eso el `typecheck` no lo detecta, porque son cadenas.

Apagado ordenado. `enableShutdownHooks()` hace que `SIGTERM` / `SIGINT` detengan el barrido de sesiones y cierren los SSE en vuelo. Un despliegue rodante no deja streams huérfanos.

Escalar. Las sesiones viven en memoria de **una** instancia. Con varias réplicas, un operador solo funcionaría contra la que le dio la sesión, salvo que el balanceador use sesiones pegajosas. Escalado horizontal real requiere un almacén compartido — ver [capítulo 19](#).

19. Limitaciones y trabajo futuro

Qué **no** hace la consola, por qué, y qué haría falta para que lo hiciera. Un proyecto que no dice dónde están sus bordes no es honesto sobre su calidad.

19.1 Limitaciones conocidas

Sesiones en memoria: una sola instancia

El almacén de sesiones es un `Map` en el proceso del BFF. Con varias réplicas, un operador solo funciona contra la instancia que le dio la sesión, salvo sesiones pegajosas en el balanceador.

Qué haría falta: un almacén compartido (Redis) o sesiones sin estado. La segunda opción es más delicada de lo que parece: haría falta cifrar el token del broker dentro de la cookie, lo que reintroduce en el navegador justo lo que este diseño saca de ahí. La primera es la correcta.

Escalado y disponibilidad

Derivado de lo anterior: la consola está pensada como **una instancia** por despliegue. No es una limitación grave —una consola de operación no es el camino caliente— pero conviene decirlo.

Sin auditoría de acciones

Quién creó, alteró o borró un topic, y cuándo, no queda registrado. Los logs del BFF registran que hubo una sesión, no qué hizo.

Qué haría falta: un log de auditoría estructurado con identidad del operador. Como la identidad viene en el JWT del broker, habría que decodificar sus *claims* — el BFF hoy no lo hace porque no necesita hacerlo, y decodificar sin verificar (no conoce el secreto) tendría que tratarse como dato no confiable.

Sin rate limiting en el login

Validar un token contra el broker es una petición barata, y el broker tiene su propia protección, pero un limitador en el BFF sería defensa en profundidad razonable.

Multi-broker no expuesto en la interfaz

La página de Ajustes muestra el destino del broker como «Confinado en el servidor (BFF)» y no permite cambiarlo. Es una **desviación consciente** de la idea original de "perfiles de conexión": permitir que el navegador reconfigure el destino convertiría al BFF en un **proxy abierto** (SSRF) y rompería el confinamiento del token.

Qué haría falta: perfiles definidos **en servidor** —una lista cerrada de despliegues conocidos, por entorno— entre los que la interfaz pudiera elegir por id. Es trabajo de servidor, no de cliente.

Los e2e no corren en el CI

El workflow llega hasta `pnpm test`. Las 21 pruebas Playwright se ejecutan en local antes de cada *push*. Un contribuidor externo que no las ejecutara no vería el fallo en el CI.

Qué haría falta: un job con `playwright install --with-deps chromium`, ejecutado en `main` o bajo etiqueta para no penalizar cada *push*.

`docker compose up` **no ejecutado**

El entorno de desarrollo no tenía Docker. El runtime de la imagen se verificó **sin el demonio** (bundle de producción correcto, servido de la SPA, deep links, `/health`, comando del `HEALTHCHECK` con exit 0/1, YAML válido), pero falta el ciclo completo en una máquina con Docker. Detalle en el [capítulo 17](#), §17.7.

Nombres de métrica: acoplamiento fuera del contrato

El `MetricsSnapshot` es una lista abierta; el OpenAPI no fija los nombres. Si el broker renombra una familia, **el `typecheck` no lo detecta** —son cadenas— y la consola degradará a «—» sin avisar por qué.

Mitigación actual: los tres módulos que fijan nombres llevan un `@see` clicable al catálogo del broker. *Qué haría falta de verdad:* que el broker publicara su catálogo de métricas como artefacto versionado y consumible (un JSON junto al OpenAPI), del que se pudieran generar constantes igual que se generan los tipos.

19.2 Dependencias del broker

Hay comportamientos que parecen carencias de la consola y son estado real del broker. La consola los degrada con honestidad; se poblarán solos cuando el broker avance:

Se ve así	Causa	Cuándo cambiará
Topología Raft sin aristas	Single-node con <code>replication_factor = 1</code> : no hay réplicas que mostrar.	Con un clúster de ≥ 2 nodos y $RF \geq 2$.
Grupos vacíos	No hay consumidores conectados.	Con consumidores reales.
<code>msg/s</code> o conexiones en «—»	El broker aún no emitía esas familias cuando se escribió el mapeo.	En cuanto las emita: los tiles se encienden solos, sin tocar la consola.

Esto último es una propiedad del diseño, no una casualidad: la consola **ya** consulta los nombres correctos y degrada solo por ausencia. Cuando el dato aparece, se muestra.

Control de ciclo de vida de nodos

Arrancar, parar o drenar un nodo desde la consola requiere una capa de servicio del SO que el broker no expone. No hay endpoint que invocar. Queda fuera de v1 por dependencia externa, no por decisión de alcance.

19.3 Trabajo futuro, por valor

Alto valor, coste contenido:

- 1. **Job de e2e en el CI** — cierra el único hueco real de la puerta de calidad.

2. **Auditoría de acciones** — necesaria en cuanto haya más de un operador.
3. **Rate limiting en el login** — defensa en profundidad barata.
4. **Alertas visuales sobre umbrales** — que la consola marque un p99 fuera de rango, en lugar de dejar que el operador lo detecte mirando. Aprovecharía los tokens de estado que ya existen.

Alto valor, coste mayor:

5. **Perfiles multi-broker definidos en servidor** — con la restricción de seguridad resuelta por diseño (lista cerrada por entorno).
6. **Sesiones en almacén compartido** — habilita el escalado horizontal.
7. **Catálogo de métricas versionado** en el broker, generable como constantes. Elimina el único acoplamiento del proyecto que no está cubierto por el compilador.

Exploratorio:

8. **Vista de consumo por topic en el tiempo** cruzando lag de grupo con throughput.
9. **Exportación de una ventana de métricas** (CSV/PNG) para incidencias.
10. **Modo quiosco** para pantallas de operación: rotación de vistas, sin interacción.

19.4 Lo que no se hará

Y por qué, para que no vuelva a discutirse:

No se hará	Razón
Almacenar series en la consola	Es el trabajo de Prometheus. Duplicarlo añade un sistema de estado que mantener y sincronizar.
Embeber Grafana	Un <code><iframe></code> no es un producto, y la visualización propia es la tesis del proyecto.
Endpoint "kill" del broker	Operación destructiva sin confirmación fuera de banda ni auditoría. No pertenece a una consola web.
App móvil nativa	La web responsiva cubre el caso; una app nativa duplica el trabajo sin añadir tesis técnica.
Escribir tipos del contrato a mano	Rompe la propiedad que sostiene todo el proyecto: que un cambio del broker falle al compilar.
Reconfigurar el broker desde el navegador	Convierte el BFF en un proxy abierto. Ver §19.1.

Apéndice A. Registro de decisiones y evolución

Las decisiones **as-built** que dieron forma a la consola, con su fecha y su motivo, y la historia de su construcción por fases. Recoge el registro que se mantuvo durante el desarrollo, para que el *porqué* de cada elección sobreviva al proyecto.

Notación: **(esencial)** = decisión con consecuencias estructurales o de seguridad; **(reversible)** = elección razonada que podría cambiarse sin rediseñar.

A.1 Decisiones de partida (2026-07-16)

#	Decisión	Tipo
1	Cliente externo, repositorio propio, contrato = OpenAPI de NexusMQ. La consola no toca el core C++; solo consume el plano de operación publicado.	esencial
2	Stack: SPA React (Vite) + BFF NestJS, monorepo pnpm + Turborepo, TypeScript de extremo a extremo.	esencial
3	Node 22 LTS , gestor pnpm .	reversible
4	Generación de contrato con <code>openapi-typescript</code> (tipos) + <code>openapi-fetch</code> (cliente ligero tipado). Datos en la SPA con TanStack Query .	reversible
5	OpenAPI vendorizado en <code>packages/contract/openapi.yaml</code> , con script <code>sync:openapi</code> que lo re-descarga del <i>raw</i> del broker. Nunca se escriben tipos a mano.	esencial
6	Estilo: Tailwind CSS + primitivos Radix UI, con sistema de color y tokens guiado por la disciplina dataviz.	reversible
7	Tests: Vitest (web, bff, contract) + Playwright (e2e de navegador) + supertest (integración del BFF).	reversible
8	Hosting v1: imagen Docker genérica + <code>docker-compose</code> de ejemplo. Sin atarse a un PaaS.	reversible
9	Transporte git = SSH. El PAT HTTPS carecía de <code>scope workflow</code> y GitHub rechazaba subir <code>.github/workflows/</code> .	reversible

A.2 El modelo de autenticación (2026-07-16)

La decisión más consecuente del proyecto, en cuatro pasos encadenados:

El broker no expone endpoint de login. El `openapi.yaml` solo define `bearerAuth` (JWT HS256), exigido si el nodo arrancó con `--jwt-secret`. **No hay ruta que acuñe tokens:** se emiten fuera del broker. (*hecho del contrato*)

Auth = «el operador pega su token». (*esencial*) En el login, el operador introduce un JWT de broker ya emitido; el BFF lo **valida contra el broker** y lo guarda **en servidor** (almacén en memoria). Al navegador solo va una **cookie de sesión** `httpOnly` con el id. El BFF **nunca conoce el secreto HS256**. El token del broker **no puede aparecer nunca** en una respuesta al navegador — y hay tests que lo garantizan.

Detección del modo por sondeo. (*esencial*) Como el BFF no conoce el secreto, descubre si el broker exige auth **sondeando su comportamiento:** `GET /api/v1/topics?size=1` **sin** token; `401` ⇒ modo secreto, `2xx` ⇒ modo abierto. Se cachea.

Cookie opaca y firmada. *(esencial)* Sesión = id aleatorio de 256 bits guardado en servidor; la cookie lleva `id.HMAC-SHA256(id, SESSION_SECRET)`, verificado en tiempo constante. El guard por petición inyecta el token confinado, o responde 401.

A.3 Tooling del BFF (2026-07-16, reversible)

NestJS 11 sobre **Express**. El paquete emite **CommonJS** (idiomático en Nest; el resto del monorepo es ESM), con DI por constructor vía `reflect-metadata`. El **proxy usa fetch nativo de undici**, no `openapi-fetch`: es *passthrough* (reemite RFC 7807 y SSE tal cual) y el contrato se usa **solo para tipos**. Config validada con **zod fail-fast**. Sesiones **en memoria**. Tests de integración con Vitest + supertest y `unplugin-swc` para la metadata de decoradores.

A.4 La reconciliación con el broker real (2026-07-17)

El error

Nombres de métrica del snapshot (decisión original, F3.1). El `MetricsSnapshot` del contrato es una **lista abierta** de series con `name` libre: el contrato **no fija los nombres**. El Dashboard asumió, en un único módulo, `nexusmq_messages_in_total`, `nexusmq_produce_latency_seconds` y `nexusmq_connections_active`, y degradaba con honestidad si faltaban. Como no había broker real a mano, se verificó contra el **doblo** — que emitía exactamente esos nombres.

La corrección

Nombres REALES del broker. *(esencial en cuanto a la fuente; reversible en cuanto al mapeo)* La fuente de verdad de los nombres es `docs/metrics.md` del broker, **no** el OpenAPI (que fija la *forma*). El broker no emite `nexusmq_*`: emite el plano de datos `nexus_broker_*` con etiquetas `{api: produce|fetch, protocol: native|kafka}` — `requests_total`, `request_errors_total`, `request_bytes_total`, `messages_total`, `request_duration_seconds` — y `nexus_broker_connections_active{plane}`. La consola codifica **contra estos nombres** y **filtra por etiqueta**. Sustituye la decisión anterior, que asumía nombres ficticios: **por eso los e2e pasaban sobre un contrato equivocado — los dobles mentían**.

Las decisiones que salieron de ahí

Gate de login `CONSOLE_REQUIRE_LOGIN`, **default true SIEMPRE**. *(esencial)* En todos los entornos. Con el flag activo, el guard exige sesión válida siempre en rutas `@Protected`, aunque el broker esté en modo abierto (deja de espejar el modo del broker). Además `GET /api/v1/metrics/snapshot` pasa a `@Protected` — antes era alcanzable sin login **incluso en modo secreto**: una fuga. El mecanismo de login no cambia; no se introduce una credencial de consola nueva.

`query_range` **con allow-list de PromQL construida en SERVIDOR**. *(esencial)* El endpoint deja de ser *passthrough* de PromQL: el cliente envía solo un **id de métrica** del catálogo (throughput, latencias p50/p99/p999, tasa de error, bytes/s, mensajes/s) más rango y `step` validados, y el **BFF construye la PromQL**. Fuera del allow-list → 400. El endpoint pasa a `@Protected`. Evita que el BFF sea un motor PromQL abierto (superficie de SSRF y DoS contra Prometheus).

A.5 Historia de desarrollo

Fase 0 — Andamiaje del monorepo

Monorepo pnpm + Turborepo con `apps/web`, `apps/bff`, `packages/contract`; TS base compartido, ESLint + Prettier, `.nvmrc` (Node 22). `packages/contract` vendoriza el `openapi.yaml`, con `sync:openapi` verificado contra descarga real y `generate` produciendo `paths / components / operations`. CI en GitHub Actions verde. Firma SSH con la identidad del autor: commits **Verified**.

Fase 1 — BFF (NestJS)

Siete ítems, cada uno cerrado con pruebas:

Ítem	Entrega
F1.1	Esqueleto Clean: seis módulos, DI por constructor, controllers finos, <code>GET /health</code> , arranque real verificado.
F1.2	Config <i>fail-fast</i> con zod (allow-list); un entorno inválido aborta con mensaje claro y exit 1.
F1.3	Proxy REST <i>passthrough</i> con undici: topics CRUD + <code>PATCH</code> , groups, cluster, snapshot, <code>healthz / readyz</code> . Validación en el borde, <code>ProblemDetailsFilter</code> global. 19 pruebas contra un doble.
F1.4	Auth con JWT confinado: login valida contra el broker, sesión en memoria, cookie <code>httpOnly</code> firmada, guard global. 7 pruebas con aserciones de no-fuga .
F1.5	Terminación SSE: latido, reconexión backoff + jitter sin tumbar el <code>EventSource</code> , timeouts de conexión e inactividad, cierre limpio. 3 pruebas con cliente SSE crudo.
F1.6	Data source de Prometheus con degradación limpia; 400 de consulta inválida, 502 de Prometheus caído. 7 pruebas con un doble.
F1.7	Servido de la SPA con fallback de deep links, sin tapar API ni observabilidad. 7 pruebas.

Fase 2 — SPA base y sistema de diseño

F2.1 Shell + tokens dataviz validados (contraste AA verificado en navegador, claro y oscuro). **F2.2** Capa de datos (`openapi-fetch` + TanStack Query), errores RFC 7807 normalizados, auth completa y primera vista con datos reales; verificada **full-stack** con el BFF real sirviendo la SPA. **F2.3** Arsenal de visualización: `useVizTokens` + cuatro wrappers (ECharts, uPlot, visx, react-three-fiber) y laboratorio `/lab`. **F2.4** Tiempo real: `useLiveStream` con fallback a polling, verificado forzando el fallo y restaurando.

Fase 3 — Vistas v1

F3.1 Dashboard vivo (derivación pura y testeada de throughput y cuantiles; dos gráficas uPlot en streaming; panel Raft). **F3.2** Topics CRUD + `PATCH` de retención **con efecto real** (el doble del broker pasa a *stateful*). **F3.3** Grupos con lag real por partición. **F3.4** Particiones con lag de réplica, cruzando describe con estado Raft. **F3.5** Cluster/Raft con **topología 3D** que responde al cambio de líder. **F3.6** Ajustes (tema, sesión, conexión).

Desviación honesta en F3.6. El criterio original decía «cambiar de perfil reconfigura el destino del BFF». **No se expuso a propósito:** el broker se fija en el despliegue (entorno confinado); reconfigurar el destino desde el cliente convertiría al BFF en un proxy abierto (SSRF) y rompería el confinamiento del token. Los perfiles multi-broker son materia de servidor.

Fase 4 — Historia y empaquetado

F4.1 Vista de Historia sobre Prometheus con modelo puro testado y degradación limpia. **F4.2** Dockerfile multi-stage no-root con `HEALTHCHECK`, `.dockerignore`, `docker-compose` de ejemplo y `deploy/prometheus.yml`. **F4.3** Hardening: cabeceras de seguridad con CSP de hash calculado del index servido, mismo origen sin CORS, validación repasada, README de despliegue.

Fase 5 — Reconciliación con el broker real

Ítem	Entrega
F5.1	<code>passThroughEnv</code> en la tarea <code>dev</code> de Turborepo: el <code>env-mode</code> estricto filtraba el entorno y el BFF abortaba.
F5.2	Dashboard e Historia remapeados a los nombres reales, con filtrado por etiqueta y agregación de histograma por <code>le</code> .
F5.3	Tile de conexiones activas con desglose por <code>plane</code> (<code>groupGaugeByLabel</code>).
F5.4	Sesiones con TTL + purga perezosa y periódica; <code>maxAge</code> de la cookie alineado.
F5.5	SSE con backpressure acotado : respeta <code>write()</code> y pausa la lectura del upstream hasta <code>drain</code> .
F5.6	<code>query_range</code> con allow-list de PromQL en servidor y <code>@Protected</code> .
F5.7	Gate de login <code>CONSOLE_REQUIRE_LOGIN</code> (default <code>true</code>) y <code>metrics/snapshot</code> protegido.
F5.8	Re-sondeo del modo del broker con TTL (60 s) y todos los dobles alineados al contrato real.
F5.9	<code>@see</code> clicable al catálogo <code>docs/metrics.md</code> del broker en los tres módulos que fijan nombres de métrica.

Cierre

Retirada del andamiaje de desarrollo, eliminación del *scaffolding* de capturas con ruta local en los specs e2e, normalización de formato con `format:check` incorporado al CI, esta documentación técnica, su PDF y la licencia.

A.6 Riesgos anotados durante el desarrollo

Se registraron mientras se construía, y siguen vigentes:

- **La topología Raft vacía y los grupos vacíos no son bugs de la consola.** Un broker single-node con `RF = 1` no tiene réplicas Raft, y sin consumidores no hay grupos. Ambas vistas deben **degradar con honestidad**, no dar error. Se poblarán solas cuando el broker avance.
- **SSE, no WebSocket.** El mecanismo es `EventSource` (una vía servidor → cliente). Las referencias de patrones sobre WebSockets valen para auth y resiliencia, pero no para el transporte.
- **Contraste y accesibilidad de las gráficas:** la validación de paleta en claro y oscuro no es opcional.
- **Sincronía del contrato:** si NexusMQ cambia el OpenAPI, `sync:openapi` + `generate`; los tipos rotos deben salir en `typecheck`, no en `runtime`. Los **nombres de métrica** son la excepción conocida —son cadenas— y por eso están anclados con `@see`.

Apéndice B. Bibliografía

Fuentes y especificaciones que sustentan las decisiones de la consola. Se agrupan por el capítulo al que dan soporte.

Especificaciones y estándares

- **RFC 7807** — *Problem Details for HTTP APIs*. IETF, 2016. Formato de error de punta a punta ([capítulo 14](#)).
- **RFC 6750** — *The OAuth 2.0 Authorization Framework: Bearer Token Usage*.
- **RFC 7519** — *JSON Web Token (JWT)*.
- **RFC 6265bis** — *Cookies: HTTP State Management Mechanism* (atributos `SameSite`, `HttpOnly`, `Secure`).
- **OpenAPI Specification 3.0.3** — contrato del plano de operación del broker ([capítulo 8](#)).
- **WHATWG HTML** — **Server-Sent Events** y la interfaz `EventSource` ([capítulo 10](#)).
- **W3C** — **Content Security Policy Level 3** ([capítulo 13](#)).
- **W3C** — **Web Content Accessibility Guidelines (WCAG) 2.2**, criterios 1.4.3 (contraste mínimo) y 1.4.1 (uso del color).

Seguridad de aplicaciones web

- **OWASP** — *Top Ten y Application Security Verification Standard (ASVS)*.
- **OWASP** — *Cheat Sheet Series: Session Management, Cross-Site Request Forgery Prevention, Server-Side Request Forgery Prevention, Content Security Policy*.
- Zalewski, M. *The Tangled Web: A Guide to Securing Modern Web Applications*. No Starch Press.
- **MDN Web Docs** — *HTTP headers, Same-origin policy, CORS*.

Arquitectura y diseño

- Martin, R. C. *Clean Architecture*. Prentice Hall, 2017. Regla de dependencia e inversión ([capítulo 5](#)).
- Martin, R. C. *Clean Code*. Prentice Hall, 2008.
- Newman, S. *Building Microservices*. O'Reilly. Origen y encaje del patrón **BFF**.
- Fowler, M. *Patterns of Enterprise Application Architecture* y el catálogo de martinfowler.com (BFF, Strangler, Circuit Breaker).
- Nygard, M. *Release It!*. Pragmatic Bookshelf. Timeouts, backoff, *bulkheads*, estabilidad.
- Wiggins, A. *The Twelve-Factor App*. Configuración por entorno, paridad dev/prod ([capítulo 18](#)).
- Procida, D. **Diátaxis** — marco de documentación técnica (referencia frente a explicación).

Sistemas distribuidos y observabilidad

- Kleppmann, M. *Designing Data-Intensive Applications*. O'Reilly, 2017. Log particionado, replicación, consenso — el dominio que la consola visualiza.
- Ongaro, D.; Ousterhout, J. *In Search of an Understandable Consensus Algorithm (Raft)*. USENIX ATC, 2014. Términos, líderes y progreso de seguidores ([capítulo 12](#)).

- **Prometheus** — *Documentation*: tipos de métrica, `rate()`, `histogram_quantile()`, `query_range` ([capítulo 11](#)).
- Tene, G. *How NOT to Measure Latency* / **HdrHistogram**. Percentiles, *coordinated omission*, por qué la media miente.
- Beyer, B. et al. *Site Reliability Engineering*. O'Reilly. Señales doradas, *probes* de liveness y readiness.

Frontend, tiempo real y visualización

- Grigorik, I. *High Performance Browser Networking*. O'Reilly. SSE frente a WebSocket, buffering de proxies, conexiones persistentes.
- **Node.js Documentation** — *Stream backpressure*. Base de `stream/backpressure.ts` ([capítulo 10](#)).
- Tufte, E. *The Visual Display of Quantitative Information*. Graphics Press. Razón dato-tinta, elementos recesivos.
- Few, S. *Information Dashboard Design*. Analytics Press. Diseño de tableros de operación.
- Brewer, C. **ColorBrewer** y Okabe, M.; Ito, K. *Color Universal Design*. Paletas categóricas distinguibles con deficiencias de visión cromática ([capítulo 12](#)).
- **Documentación de las librerías empleadas** — React, Vite, NestJS, TanStack Query, Zod, undici, uPlot, ECharts, visx, react-three-fiber, Radix UI, Tailwind CSS, Playwright, Vitest.

Proyecto complementario

- **NexusMQ** — broker de mensajería distribuido en C++23 que esta consola administra: <https://github.com/AOjeda006/NexusMQ>. Su documentación técnica (`docs/tecnica/`) y su catálogo de métricas (`docs/metrics.md`) son la referencia aguas arriba de los capítulos [8](#) y [11](#).